Empirical Software Engineering (2025) 30:79
https://doi.org/10.1007/s10664-024-10557-2

User feedback in continuous software engineering: revealing the state-of-practice

Anastasiia Tkalich[1] · Eriks Klotins[2] · Tor Sporsem[1] · Viktoria Stray[1,3] · Nils Brede Moe[1,2] · Astri Barbala[1]

Accepted: 30 September 2024 / Published online: 7 March 2025

Abstract

Context Organizations opt for continuous delivery of incremental updates to deal with uncertainty and minimize waste. However, applying continuous engineering (CSE) practices requires a continuous feedback loop with input from customers and end-users.

Challenges It becomes increasingly challenging to apply traditional requirements elicitation and validation techniques with ever-shrinking software delivery cycles. At the same time, frequent deliveries generate an abundance of usage data and telemetry informing engineering teams of end-user behavior. The literature describing how practitioners work with user feedback in CSE, is limited.

Objectives We aim to explore the state of practice related to utilization of user feedback in CSE. Specifically, what practices are used, how, and the shortcomings of these practices.

Method We conduct a qualitative survey and report analysis from 21 interviews in 13 product development companies. We apply thematic and cross-case analysis to interpret the data.

Results Based on our earlier work we suggest a conceptual model of how user feedback is utilized in CSE. We further report the identified challenges with the continuous collection and analysis of user feedback and identify implications for practice.

Conclusions Companies use a combination of qualitative and quantitative methods to infer end-user preferences. At the same time, continuous collection, analysis, interpretation, and use of data in decisions are problematic. The challenges pertain to selecting the right metrics and analysis techniques, resource allocation, and difficulties in accessing vaguely defined user groups. Our advice to practitioners in CSE is to ensure sufficient resources and effort for interpretation of the feedback, which can be facilitated by telemetry dashboards.

Keywords Continuous software engineering · User feedback · Software product · Continuous experimentation · Data-driven product development

1 Introduction

Companies developing market-driven, software-intensive products and services face ever-increasing pressure to rapidly deliver high-quality, cutting-edge solutions. A viable approach

Communicated by: Bram Adams

Extended author information available on the last page of the article

is to adopt continuous software engineering practices. Continuous software engineering (CSE) aims to deliver small software increments in short and frequent cycles, that is, continuously (Fitzgerald and Stol 2017; Klotins et al. 2022; Humble and Kim 2018). This unlocks the opportunity to collect user feedback for each delivery, informing developers on how users react to the delivered code.

Numerous studies show that genuine user feedback is critical for informing product development and is instrumental to project success in software engineering in general (Gallivan and Keil 2003; Maalej et al. 2009; Fabijan et al. 2015; Guzman et al. 2017) and agile development in particular (Paetsch et al. 2003; Brhel et al. 2015). In this paper, we use the term *user feedback* loosely to characterize all the feedback and data collected from current and potential end-users, their proxies, and similar sources for the purpose of product development.

Although in principle it is clear how to collect user feedback in general, it is not clear how to efficiently and effectively integrate it into the CSE process. Increasing the cadence of software delivery requires a matching increase in the collection and analysis of user feedback. This creates new challenges for developers and their methods. Arranging meetings with stakeholders for feedback elicitation activities, for example, interviews, takes time and can be seen as a bottleneck for frequent software delivery (Fabijan et al. 2015; Humble and Kim 2018). Furthermore, stakeholders may not be prepared to participate in increasingly more frequent elicitation meetings (Inayat et al. 2015). To mitigate these challenges, companies can opt for quantitative methods (such as telemetry and experimentation) that allow learning about user preferences relatively quickly and seamlessly for users (Auer and Felderer 2018; Fabijan et al. 2015). However, quantitative feedback alone offers limited insight into the motivations and reasoning of users (Inayat et al. 2015).

Some work has been done to understand the practices related to user feedback in CSE. Johanssen et al. (2019) explore how various feedback techniques are practiced in CSE and with what consequences, and propose recommendations on how to incorporate user feedback. Yaman et al. (2020); Lindgren and Münch (2016); Fabijan et al. (2017); Ros et al. (2024) study user involvement in experiment-driven software development. The experimentation approach treats product requirements and roadmaps as mere hypotheses and suggests designing quantitative experiments to verify them with user behavior. Melegati et al. (2021) summarize activities related to hypothesis engineering for continuous experimentation.

Despite these insights, companies struggle to solve the practical challenges of combining user feedback and CSE. By providing further empirical examples of practices and uncovering practical challenges, we aim to contribute with a more nuanced picture of how user feedback is handled in various contexts of CSE. Furthermore, our ambition is to propose an integrated conceptual understanding of how user feedback relates to the CSE process.

Therefore, our study aims to explore *how user needs and preferences are inferred and what are the challenges related to user input in CSE.* We report a qualitative survey of 13 software-intensive products developed following CSE practices (Jansen et al. 2010). Our results are based on 21 interviews with various stakeholders involved in product development and utilization of user feedback.

Our findings provide an overview of the capture and analysis of user feedback in CSE. Based on these findings, we discuss how the benefits of user feedback in the CSE settings can be maximized. The study makes the following contributions:

1. Provides a detailed empirical description of the utilization of user feedback and user data in 13 industrial cases using CSE.
2. Gives an overview of the challenges related to the utilization of user feedback and potential mitigation strategies.

3. Based on earlier work, presents a conceptual model of utilizing user feedback in CSE.
4. Provides practical recommendations on how the benefits of user feedback may be maximized in CSE.

The remainder of this paper is organized as follows. First, we review the literature on CSE and user feedback, including typical challenges in Section 2. In Section 3, we describe our research approach, introduce the industrial cases, and discuss the main threats to the validity of this study. Next, in Section 4, we report the results related to the state of practice identified in the cases. In Section 5, based on our findings and related work, we discuss how to maximize the benefits of user feedback in CSE.

2 Background and Related Work

2.1 Continuous Software Engineering

Continuous Software Engineering (CSE) originates from lean and agile principles in software engineering (Fowler et al. 2001; Poppendieck and Poppendieck 2003; Fitzgerald and Stol 2017). Agile software engineering emphasizes the delivery of working software over upfront planning, documentation, rapid response to change, and collaboration versus negotiation (Fowler et al. 2001). These principles have inspired many agile practices to deliver value in small increments and to eliminate activities that do not contribute to the delivery of software (Jalali and Wohlin 2012; Martin 2002; Sidky et al. 2007). Lean principles aim to maximize value delivery, minimize waste, and anticipate changes in user needs (Rodríguez et al. 2012; Alahyari et al. 2019).

The CSE paradigm is similar to lean and agile in that it attempts to maximize value delivery, enable flexibility, and minimize waste by frequently delivering software to end users in small increments. Fast cadence is achieved by extensive end-to-end automation, tools, automated tests, and staging environments, that is, the pipeline, which allows software deliveries to end users several times per day (Forsgren et al. 2019; Humble and Kim 2018; Fitzgerald and Stol 2017).

A key distinction of continuous engineering is that, in both agile and plan-driven contexts, recently finished software is shelved until the end of the sprint or other predefined release date. While shelved, the software does not generate value and only contributes to waste. CSE addresses this by ensuring that the latest software version is delivered instantly and automatically to end users (Humble and Kim 2018).

A crucial element of CSE, as with any other development methodology, is the collection and application of user input. In CSE, frequent software delivery should match equally frequent user input. Small and frequent deliveries allow for continuous experimentation (Fabijan et al. 2017), collection of telemetry and data (Maalej et al. 2015), and the determination of what exact changes in software caused the changes in user behavior. The use of data is a crucial principle in CSE, allowing frequent and informed adjustments in the direction of the product, thus minimizing waste and building software that more closely matches user needs (Humble and Kim 2018; Klotins et al. 2022).

The application of qualitative requirements elicitation and validation techniques is challenging in CSE, especially when developing market-driven products and services. For example, the rapid cadence of software delivery leaves little room to schedule user interviews. Conducting interviews with a few out of a million users can be perceived as wasteful

in favor of quantitative approaches, such as A/B tests and analysis of user telemetry (Regnell and Brinkkemper 2005).

Bosch et al. (2018) argue that in dynamic, market-driven contexts, the best results can be achieved by combining traditional requirement-driven approaches with data-driven practices and the use of machine learning and artificial intelligence. This combination contributes to continuous end-to-end flow between user insight, business strategy, and software development, which is the foundation of CSE at the organizational level (Fitzgerald and Stol 2017). Tkalich et al. (2022) found that product management is a discipline that facilitates this flow, by formulating and testing the hypothesis in close collaboration with both the users and the development team.

2.2 The Importance of Feedback in Continuous Software Engineering

An essential component of CSE is continuous improvement. The CSE process aims to improve both the software delivery process and the software itself (Poppendieck and Poppendieck 2003; Fitzgerald and Stol 2017). In our earlier work, see Klotins et al. (2022), we present a conceptual model of continuous software engineering. The model comprises a closed loop; see Fig. 1.

The upper part of the model in Fig. 1 focuses on delivering software by designing product plans, implementing them with the help of automation, and deploying the latest changes in operations. However, the bottom part of the model focuses on gathering user feedback and telemetry, monitoring it, and interpreting it for use in the next cycle.

Hence, the delivery of the software and the collection of meaningful feedback to guide further development are integral to each other. Importantly, our previous work shows that

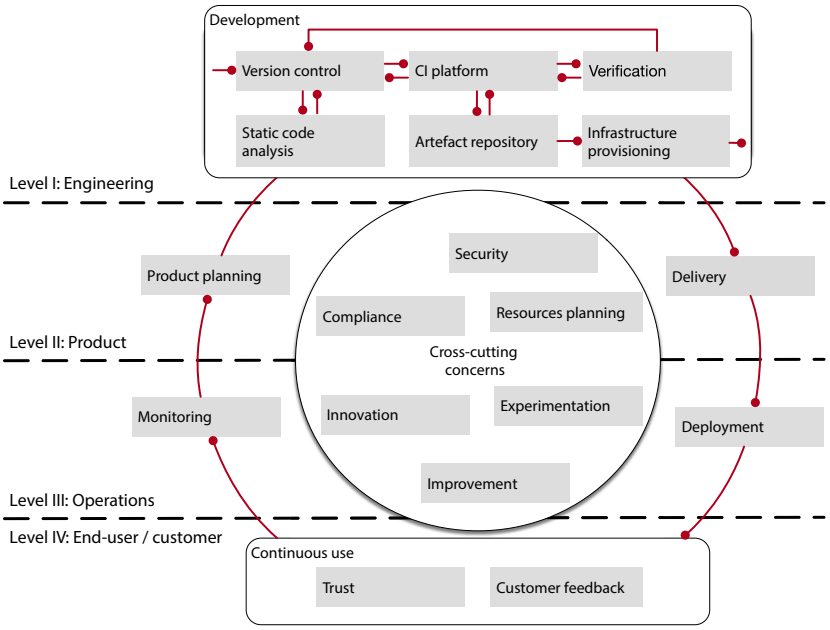

Fig. 1 The conceptual model of CSE (from Klotins et al. 2022)

companies often opt for low-hanging fruits to increase the pace of software delivery. However, without considering the whole cycle, and especially the ability to collect and interpret user input on recently delivered software, companies could waste resources and achieve suboptimal results (Klotins et al. 2022; Klotins and Gorschek 2022).

2.3 Implicit and Explicit Feedback

In this study, we use the term user feedback to characterize all feedback and data collected from current and potential end-users and their proxies (such as data reflecting the user property / behavior, etc.).

The literature differentiates between *explicit* and *implicit* user feedback (Fabijan et al. 2015; Maalej et al. 2015). In very general terms, explicit feedback allows one to evaluate subjective opinions of users, while implicit feedback summarizes their objective opinions (e.g., how and when they actually use the software and its features) (Maalej et al. 2015). However, a further differentiation can be made:

Explicit Feedback results from techniques in which users are aware of sharing their feedback with developers to change the product. This feedback is intentionally provided by users and summarizes their subjective opinions (Maalej et al. 2015). This feedback may be based on the active participation of users or face-to-face interaction with product developers, for example, user interviews, questionnaires, and surveys (Fabijan et al. 2015).

Implicit Feedback results from techniques in which the users are *not* aware of sharing their feedback with the developers for the purpose of changing the product. This can be automatically collected information about software usage (Maalej et al. 2015) that reflects the users' actual behaviors while interacting with the product. Although in many cases users know that their data are being automatically collected (e.g., social media activity), they are not always aware how it is used, thus making the feedback provided in this way implicit. Most implicit feedback is used to support inquiries and bug fixes, not to discover new requirements or create new features (Johanssen et al. 2019).

Earlier studies reported that companies may struggle to combine explicit and implicit user feedback despite its apparent advantage of combining deep understanding and generalizability (Ghasemi and Amyot 2020; Franch et al. 2021).

2.4 User Feedback and Data Collection Techniques

Based on the earlier literature, Fabijan et al. (2015) summarized various techniques for capturing user input. Some of them, such as interviews and prototype testing, are called user feedback techniques and are likely to result in explicit feedback. In contrast, user data collection techniques (such as event data and online ads) often provide implicit feedback and are based on various quantitative data sources about the user (demographic, behavioral, etc).

The overview of user feedback and data collection techniques is provided in Table 1. In the table, we define each technique based on the work of Fabijan et al. (2015) and differentiate between implicit and explicit feedback techniques according to the categorization suggested by Maalej et al. (2015).

Although each of the techniques in Table 1 is categorized as either explicit or implicit, the feedback they produce is not always binary. For example, the output of *observations* is categorized as explicit feedback because in most cases users are aware that they are being

Table 1 Techniques for capturing user input (user feedback and data collection techniques)

	Technique	Type of feedback	Definition and further references
1	Interviews	Explicit	Users answer developers' questions face-to-face
2	In-product surveys	Explicit	Feedback that allows evaluating the users' appreciation of the current product
3	Observations	Explicit	Developers observe the users (Johanssen et al. 2019).
4	Theater sessions	Explicit	Several users express feature requests and provide inputs on how features could be used in their context
5	Prototype testing	Explicit	Evaluation of partially developed interfaces and functionality in collaboration with users (Johanssen et al. 2019)
6	Incident reports	Explicit	Descriptions of problems generated by users (e.g. support data, trouble tickets) (Johanssen et al. 2019)
7	Developers as users	Explicit	internal evaluation of the product by the developers
8	User pairing	Explicit	Facilitated discussion between several users simultaneously
9	Walk-throughs	Explicit	Integration exercises performed by developers to verify that all user scenarios work; the results are presented in front of the customer representatives
10	Online ads	Implicit	displaying the product information on external sites that allow users to evaluate their interest in potential new products (e.g., evaluating the number of users who followed the ads)
11	Beta testing	Implicit	an early version of the product is given to a sample of users (Johanssen et al. 2019)
12	Operational and event data	Implicit	data collected online per user (e.g. demographic data, behavioral data) and data on the product's performance (responsiveness, errors, etc.)
13	A/B testing	Implicit	displaying different versions of the same feature to evaluate the version that produces the desired response (Bosch et al. 2018; Yaman et al. 2020)
14	Social networks	Implicit	evaluating the product on social media platforms (Bajic and Lyons 2011; Guzman et al. 2017).
15	Crowd-funding platforms	Implicit	evaluating the success rate of a product based on community support

The list of techniques and definitions is based on the work by Fabijan et al. (2015). The references indicate other literature that focuses on specific techniques

observed (e.g., due to legal regulation requiring informed consent). However, it is possible to imagine scenarios in which users are unaware that they are being observed. Think of someone developing a screen time control application who observes potential users in the train with the aim of discovering how often they use their mobile devices. In the same way, *social networks* can result in both implicit and explicit feedback depending on the intentions of the user (sharing their opinion about the product with the general public or sharing opinion with the product developers). However, we chose to categorize this entry as "implicit" because this type of feedback was reported as the outcome of applying social networks to our data.

Finally, the entries in Table 1 differ essentially from each other and can be further categorized. For example, *prototype testing* and *beta testing* are essentially not independent techniques but a combination of an artifact and other techniques. These techniques require evaluations of artifacts (such as prototypes or beta versions of a product) with follow-up techniques such as interviews, surveys, observations, and so on. In addition, *developers as users* is essentially a sampling technique. In this case, the same procedures can be applied to developers as to regular users (e.g., interviews, prototype testing, etc.), the only difference being how the users are selected (based on randomness or purposefully). Finally, A/B testing boils down to data analysis and does not necessarily have any specific procedure for collecting the user data. We will return to the distinctions between different types of feedback techniques in the description of the results (Section 4.1).

Lessons learned from open-source ecosystems reveal complex communication patterns to exchange feedback between different groups. For communities to benefit, feedback must be synchronized, heterogeneous, and include mechanisms to incentivize them. At the same time, collecting diverse feedback from multiple sources is likely to break the synchronization of the feedback. Hence, there is a trade-off between collecting more diverse and informative feedback and synchronizing the feedback with the development cycles (Foundjem et al. 2023).

2.5 Challenges Related to Continuous User Feedback

Earlier literature has identified numerous challenges related to continuous user feedback. In their study of 12 companies involved with CSE, Ros et al. (2024) found that although the CSE approach may reduce the risk of wasting resources developing features of little value to users, there are many challenges at stake. For example, CSE is mainly suitable for user-intensive and user-facing software companies, but only if combined with other software improvement strategies (Ros et al. 2024). Another finding of this study was that user data volume is critical for CSE throughput, as low user data volumes meant long lead times for experiments and hence lack of speed.

Several other challenges have been identified. First, access to end users and usage data can be limited due to legal concerns, which is especially typical for B2B contexts where users are stakeholders in the customer company (Lindgren and Münch 2016; Yaman et al. 2020; Klotins and Gorschek 2022). Second, some ways of collecting data can be time-consuming to implement. For example, questionnaires, interviews, and other types of user feedback techniques are time-consuming (Fabijan et al. 2015).

Using secondary data is complicated. For example, interpreting reviews from the App Stores requires manual effort and additional user context (Franch et al. 2021). In addition, companies may face challenges in analyzing user data and formulating analysis results as

action points for further product planning (Lindgren and Münch 2016; Johanssen et al. 2019). This can happen due to insufficient competence in data analysis or due to the vast diversity of information and its aspects available to users, such as the meaning of user metrics, data quality problems or method concerns (Lindgren and Münch 2016; Fabijan et al. 2017).

Thus, selecting a suitable set of inputs to assess end-user behavior and product success can be challenging. This may prevent product teams from making good decisions about bringing the product forward (Lindgren and Münch 2016; Fabijan et al. 2017; Bosch et al. 2018).

Other challenges related to user feedback in CSE include lack of time and funding, slow release cycles, lack of agile and transparent organizational culture, and inadequate analysis of user data, leading to a reduced ability to formulate the backlog (Lindgren and Münch 2016). Challenges may also be related to the way user feedback is captured and how it is captured and analyzed. For example, Johanssen et al. (2019); Bajic and Lyons (2011) found that developers prefer explicit feedback over implicit feedback and that the feedback is used mainly for verification of requirements rather than validation, which shifts the focus from exploration of the actual needs of the user to adequate representation of the specifications. Other findings indicate that implicit feedback is preferred (Johanssen et al. 2019; Yaman et al. 2020). In addition, developers tend to rely on manual feedback capture and are less likely to use automated tools, which reduces the efficiency of the development process (Johanssen et al. 2019).

Finally, challenges might arise when the user data is collected through hypothesis-based experiments. Fabijan et al. (2015) identify several challenges when studying such experiences at Microsoft. For instance, in early-stage experiments, the instruments (e.g., telemetry, event logs) might not be of good quality, leading to uncertainty about the experiment's results. Furthermore, advanced experiments require a high level of maturity in different dimensions, including technical, organizational, and business-related aspects. Sometimes, a dedicated experimentation platform may be required to setup, evaluate, and follow-up A/B tests. In addition, data scientist expertise is needed to set up and analyze the experiment data, deploy specific metrics, and share the data to reuse throughout the company. Finally, business metrics are essential for evaluating and setting expectations for particular products, which need to mature along with the maturity of the product.

3 Research Method

We will now describe the aim of our study and the research questions, our overall research approach, the context of our study, data collection, and how we analyzed the data.

3.1 Aim and Research Questions

Our study aims to explore the state of practice of utilizing user feedback in CSE. Specifically, our focus is to understand what data collection practices are used, to what end, and the shortcomings of the applied practices. To guide our study, we suggest the following research questions:

RQ1: How is user feedback collected and applied in continuous software engineering?
Motivation: With this research question, our objective is to explore the state of practice of collecting user feedback in the context of CSE. Significant knowledge has been accumulated about user feedback and data collection techniques in requirements engineering (Bajic and

Lyons 2011; Brhel et al. 2015; Fabijan et al. 2015; Guzman et al. 2017). However, the extent to which these practices apply in CSE is mostly unexplored.

RQ2: What are the challenges related to utilizing user feedback in CSE?
Motivation: With this research question, our aim is to identify the challenges of collecting user feedback in the CSE context. Earlier work, for example, Lindgren and Münch (2016); Fabijan et al. (2017); Johanssen et al. (2019); Yaman et al. (2020); Klotins et al. (2022) point to potential challenges. However, the extent of these challenges and their relationship to specific feedback collection techniques or elements of the CSE process have not been explored sufficiently.

3.2 Research Approach

To meet our goal of reporting the state of practice in industry, we opt for a flexible and qualitative research design and use a qualitative survey (Jansen et al. 2010). A qualitative survey aims to explore the diversity of a phenomenon in a sample. Jansen et al. (2010), a qualitative survey is close to both case study and grounded theory. A systematic data collection from the cases permits two-dimensional, case-by-case, and cross-case analysis, thus allowing several empirical cycles (research question-data collection-analysis-report). Qualitative surveys have been successfully applied in the field of software engineering (Lindgren and Münch 2016; Melegati et al. 2020; Klotins et al. 2019).

We motivate our choice of qualitative survey method by the need to describe and explore the practices related to user feedback. Specifically, our goal is to assert practices related to user feedback in our sample and explain why these practices are preferred in each context. Thus, the purposes of our survey can be described as *descriptive* and *explanatory*, as described by Wohlin et al. (2012).

Our specific units of analysis are software products (cases) embedded in the companies studied.

We focus more on exploring variations in the state of practice of user feedback and less on in-depth analysis of each case, which differentiates qualitative surveys from case studies (Jansen et al. 2010). Moreover, unlike quantitative surveys, we do not seek generalization of our results (Jansen et al. 2010). Instead, we rely on the explanatory power of qualitative data.

3.3 Study Context

This study was conducted in the context of three larger research projects: 10xTeams, TransformIT, and Digital Class (see Acknowledgments), where several companies introduced CSE in their product development process. We acquired access to the products (cases) because the companies were part of these research projects. All studied cases are based in companies with offices in Norway. Detailed descriptions of the cases and the respective companies are provided in Appendix I, where we also refer to earlier publications based on some of the cases and companies.

We leveraged the contact network with our industrial partners to identify cases that relied on CSE. Hence, our sampling approach combines accidental and snowball sampling (Linaker et al. 2015). This approach led us to collecting the accounts of 19 practitioners from 13 products.

3.4 Data Collection

Data collection was carried out between September and December 2022. Semi-structured interviews were our main data collection technique. Each interview was conducted with a minimum of two authors participating in each interview. In such a way, we attempted to reduce the confirmation bias. During the interviews, one interviewer led the interview (asking the most questions), whereas the other took notes and asked additional questions / clarifications.

The interviews were carried out as video calls (Microsoft Teams) or on-site with an average length of 45 minutes ranging between 32 and 63 minutes per interview. The interview recordings were subsequently transcribed, resulting in 356 pages of transcripts (an average transcript corresponded to 24 pages per 1-hour recording). When available, we also collected materials related to each of the products (e.g., accessed the applications/websites, video recordings of the tools in use, photos of the whiteboards, etc.). In interviews, we asked informants to tell a story about their products, which feedback techniques they apply and why, and which challenges they encounter with respect to these techniques (see Table 2). We used a large number of follow-up questions to ensure that the interviewees provided rich descriptions and that we fully understood the answers. Since the follow-up questions were different from case to case, only the main questions are shown in the interview guide (Table 2).

3.5 Case Summary

The cases in this study are software products that rely on CSE. Data collection resulted in a total of 21 interviews from 13 cases. We summarize the cases that we studied in Table 3. In the table, we denote each case with a case ID (Px) and use it to maintain traceability between our results, conclusions, and cases. Whenever the real name of the product is available, it is stated in parentheses.

The column "Product description (maturity stage)" describes the essence of the product and how far the product came in its journey. We categorized product maturity to better indicate differences between cases. We had formulated the definitions of the maturity stages and subsequently validated them with our informants. We identified three stages:

1. Early stage - a stage between the first product release and readiness for scaling. The product team has built the product's first version and released it to the first customer.

Table 2 Interview guide items

#	Interview guide item	Research focus and RQs
1	Tell a story of your product	Product and market context
2	What kind of user data do you use for developing your product? How do you use these data? Can you give an example of a change made in the product based on the user data? What kind of tools/practices do you use to ensure the end-user insight?	User feedback and data collection techniques (RQ1)
3	Why do you use these data/did you make the change)?	Rationales behind applying the feedback techniques (RQ1)
4	What works well/not so well?	Challenges related to user feedback (RQ2)

Table 3 Summary of the studied cases and the data collected per case

Case ID (*Product name*)	Product description (maturity stage)	User segments (Niche/General)	Market type	Interviewed roles (n. of interviews)
P1 (*Vake*)	AI-based maritime surveillance (early-stage)	Coast guard officers (N)	B2B	Product manager (1)
P2 (*Hjernelæring*)	Online mental health guide for parents and teachers (early-stage)	Parents, school teachers, children (G)	B2B & B2C	Product manager (1)
P3 (*Unicad*)	Online engineering calculation tool (early-stage)	Construction engineers (N)	B2B & B2C	Developer, Product manager (1)
P4 (*Byks*)	Online sports coaching (early-stage)	Coaches, athletes (G)	B2B & B2C	Product manager (1)
P5 (*Flow Technologies AS*)	Online rehabilitation support (growth-stage)	Patients, rehabilitation institutions (N)	B2B	Data scientist, Product manager (2)
P6 (*Noba*)	Online nutrition support (growth-stage)	People with IBS[a] -tendencies (N)	B2C	Product manager (1)
P7	Online banking service (mature-stage)	Bank customers (G)	B2C	Tech lead, Product Owner (3)
P8	Online public services case processing (early-stage)	Public servants (N)	N/A	Product manager, lawyer, developer (3)
P9	Online recruitment platform (mature-stage)	Recruiters, Candidates (G)	N/A	Team lead, product owner (2)
P10 (*Emissions Insights*)	Shipping vessel emissions tracker (early-stage)	Ship operators (N)	B2B	Product manager (2)
P11	Online journey planner (mature-stage)	Travelers (G)	B2C	UX-designer (2)
P12 (*Sandstorm*)	Online meme generator (growth-stage)	Users of memes (G)	B2C	Product manager (1)
P13	Online marketplace for designers (growth-stage)	Architecture designers, design consumers (N)	B2B & B2C	Product manager (1)
	Total interviewees (interviews)			19 (21)

Real names of the products are stated in parenthesis (whenever available)
[a] Irritable bowel syndrome

However, there is still a need to better understand what problem the users solve with the product and how the user problem is solved (Ros et al. 2024). The aim of this stage is readiness for scaling.
2. Growth stage - the product is ready for scaling, attracting additional users/income without adding extra effort to the team (Crowne 2002). Therefore, the main focus is on marketing and sales, while the engineering team should cope with a continuous flow of user requirements and product variations for different markets. The aim is to achieve the desired market share and growth rate. In non-commercial products, this stage is about acquiring users and improving their level of satisfaction.
3. Mature-stage - the product and the main customer base are established, and the aim is to preserve the established market share and optimize its operations.

The column "User segment" describes the end-users that the products aim at and whether these users can be described as the general public (e.g. bank customers, travelers, or parents) or specific niche groups (such as construction engineers and ship operators).

In the column "Market type", we differentiate between products aimed at consumers (B2C) and businesses (B2B). Some products were aimed at both. For example, P2 is a tool aimed at both parents (B2C) and schools (B2B). In some cases, such categorization was not applicable because they belong to the public sector and are not meant for any business.

Finally, the column "Interviewed roles" indicates the qualitative data collected per product. We interviewed 1-3 stakeholders per case. The number of interviewees depended on their interest in participating in our study. In our analysis, we normalize the results per case. We count how many times a phenomenon was mentioned per case, not per interview.

3.6 Data Analysis

We conducted data analysis in two main steps: 1) analyzing within-case data and 2) searching for cross-case patterns (Eisenhardt 1989). This approach implies that the researchers familiarize themselves with each case before exploring their similarities and differences. After these steps were completed, we contacted the interviewee to validate the preliminary results, which was the final stage of the iterative data collection process. In doing so, we hoped to increase the validity of our findings. At each step, multiple researchers continuously discussed and cross-validated the data analysis by checking agreement and interpretations. In this way, we were trying to reduce confirmation bias (hear what we wanted to hear from the informants). We continue by describing each data analysis stage in detail.

Analyzing Within-Case Data At this stage, we applied thematic analysis (Braun and Clarke 2006) to acquire an overview of the data. To perform the thematic analysis, we used the NVivo tool (version 13). The thematic analysis was performed by the first and the third authors. Inspired by the recommendations on the thematic analysis, we followed a four-step procedure to identify the themes in the transcripts:

1. Familiarizing ourselves with the data - we read the transcripts thoroughly to recap the cases. We were familiar with the case companies from before, which made capturing the products and their context easier for us.
2. Generating initial codes - The transcripts were coded line by line to identify all the meaningful content, which resulted in 766 initial codes.
3. Searching for themes - All the codes were grouped as initial themes to summarize the important information per case.

4. Reviewing the themes - The first two authors reviewed the themes. The list and the definition of user feedback and data collection techniques from Fabijan et al. (2015) were used for uniforming the techniques across the sample. At this stage, themes such as interviews, telemetry dashboards, and developers as users were established. When the initial codes did not match the description from the list in Fabijan et al. (2015), we introduced new theme names. The themes related to challenges in CSE were also defined at this stage.

See Table 4 for examples of the thematic analysis procedure. For an overview of the initial themes, sub-themes and the underlying codes, please consult the supplementary material online [1].

Searching For Cross-Case Patterns The cross-case analysis was performed to compare the codes related to the feedback techniques (RQ1) and the challenges (RQ2). For this purpose, we used a spreadsheet to identify the differences and similarities across the cases regarding RQ1 and RQ2 and the variables from the case context, such as company size, market type, and end-user type. The themes related to the feedback techniques were grouped according to their function: feedback collection, sampling; and analysis and presentation.

Validation of the Preliminary Results We shared the preliminary results (the write-up and the summary tables) with our interview participants. We asked them to provide their feedback in case we misunderstood something or lacked significant details. Out of the representatives from 13 cases, we received responses from 9. All their comments were taken into consideration when writing the final report.

4 Results

We will now describe our findings on user feedback collection and application (RQ1) and challenges related to utilizing this feedback (RQ2) in CSE. Section 4.1 describes the summarizes the techniques for capturing and interpreting user feedback and the goals for applying these techniques. Section 4.2 summarizes the challenges related to user feedback in CSE.

4.1 User Feedback Techniques

User feedback techniques are used to elicit implicit and explicit end-user feedback. The selection of feedback collection techniques determines the suitable analysis and interpretation techniques.

We will now describe each of the techniques separately. Each description contains an overview of how the technique was applied and the goal(s) of its application. The descriptions of the goals are included because understanding them was often crucial to understanding the essence of the techniques. All the goals for each technique are further summarized in Table 5. For a better overview, in the upcoming subsections we use the indices **Gx** to mark each of the goals after each mention.

[1] https://zenodo.org/records/12544288

Table 4 Thematic analysis: illustration of the coding process

Excerpt from an interview transcript	Initial code	Initial theme	Reviewed theme
"We, developers, can easily discover a bug while using the mobile bank: "Oh, here is a bug, I can fix it tomorrow. " (P7)	Dogfooding - one can discover a bug while using one's own application	Types of user feedback	Developers as users
"We have had the metrics from day one [...] Moreover, we have a metric for all the functionality and how it is used. If we see that some numbers do not give much insight, we can remove some metrics and add others. The dashboard cannot be static when the product, the business, and the customer change all the time". (P5)	Established the metrics from day 1 and continuously updated them since.	Metrics	Telemetry dashboards
	If the data show that some functionality is not in use, it is removed		
"We can ask the data team through a developer in our team [...], but then we need to know what to ask for and whom to ask." (P11)	Cumbersome process of getting usage data from the data team	Acquiring data from others	Challenges - metric-related
"It would be very nice to have all the numbers on how the users interact with the service somewhere on Amplitude, but this is not something that we have today." (P4)	Wish to have more insight into usage data	No established dashboard	Challenges - resource-related, Telemetry dashboards

Table 5 Summary of goals for collecting user feedback in the studied cases

ID	Goals	User feedback techniques							
		Interviews	In-product surveys	Observations	Prototype testing	Incident reports	User requests/ suggestions	Online ads	Beta testing
G1	Understand the user's context	•		•	•				
G2	Interpret implicit feedback	•							
G3	Devise an optimal solution given resources constraints	•							•
G4	Build relationships	•		•					
G5	Determine the user's reaction to implemented features		•	•	•				•
G6	Determine if the functionality works as expected					•			•
G7	Determine user's interest	•						•	
G8	Find design with highest user engagement								
G9	Reveal user's preferences						•		
G10	Determine user's willingness to pay								
G11	Plan the development process				•				

Table 5 continued

ID	Event data	Social networks	Sales data	Convenience sampling	Developers as users	A/B testing	Telemetry dashboards	Cross-validating feedback
G1				•	•			•
G2								
G3				•	•			
G4								
G5	•				•		•	•
G6	•			•	•		•	
G7		•	•				•	
G8						•		
G9								
G10			•					
G11							•	

To identify the user feedback techniques, we relied on a list of techniques reported by Fabijan et al. (2015) as a starting point. Tables 6, 7, and 8 show the techniques identified in this study. The techniques summarized by Fabijan et al. (2015) are shown in regular text, whereas bold text highlights additional techniques identified in our results.

User feedback techniques comprise explicit user feedback techniques (Section 4.1.1) and implicit user data collection techniques (Maalej et al. 2015) (Subsection 4.1.2. In addition, our analysis indicated that user sampling can be used as a feedback technique (subsection 4.1.3. Finally, we differentiate a group of techniques aimed at data analysis and presentation (subsection 4.1.4).

4.1.1 Explicit User Feedback Collection Techniques

These are techniques in which the users are aware of sharing their feedback with the developers for the purpose of changing the product.

The explicit user feedback techniques described by our informants were user interviews, in-product surveys, observations, prototype testing, incident reports; and user requests and suggestions (see Table 6).

User Interviews is a technique when users answer developers' questions face-to-face (Fabijan et al. 2015). In today's post-COVID-19 reality many interviews happen through video call. User interviews were among the most popular user feedback techniques mentioned in 11 cases; see Table 6.

Our data provide examples of informal interviews, e.g., phone calls aiming to capture the users' current business goals (P4), just as structured interviews The techniques was often combined with prototype testing (the users are asked to comment on how they perceive the prototype).

The purpose of user interviews was to gather first-hand information on how users perceived the product, allowing better understanding of the context of users **(G1)**. Meanwhile, there was a difference between the types of private market (B2C) and corporate (B2B) regarding why the interviews were applied.

For B2C products the interviews provided a better understanding of generic user's needs and what triggers such users' interest **(G7)**. For example, the P2 team explored how users discovered the product, which actions they took to interact with it, and whether they continued to engage with it. Specifically, the use of the so-called AARRR framework was reported to structure the interview guide. AARRR stands for Acquisition, Activation, Retention, Referral, Revenue, which refers to user behavior metrics that the products should be monitored by Plan (2024). The product manager of P2 reported using these metrics to formulate questions for users on specific product dimensions.

For the B2B products the purpose of the interviews was often to build relationships with contacts on the customer's side **(G4)**. Such relationships were described as a foundation for subsequent mutual trust and, hence, the ability to better understand the user's needs and intentions. Face-to-face interactions created opportunities for bonding and were likely to increase the user trust to the product developers, and hence share more feedback. For example, the product manager of P4 arranged phone calls with two pilot sellers on the training platform to informally discuss their life, their businesses, and what they want to achieve by selling their training on P4. As pointed out by this quote, such informal touch points appeared to facilitate honest feedback:

Table 6 Feedback techniques identified in the studied cases

ID	Feedback technique	Cases												
		P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13
T1	User interviews	•	•	•	•	•	•	•	•	•	•			•
T2	In-product surveys				•		•	•						
T3	Observations		•	•			•		•			•		
T4	Prototype testing	•	•	•	•	•		•		•	•	•		
T5	Incident reports (user-generated)				•	•		•	•		•			
T6	**User requests**	•					•		•			•		•
T7	Online ads		•		•									
T8	Beta testing		•		•	•	•	•			•			
T9	Event data (incl. automated incident reports)			•	•	•	•	•	•	•		•	•	•
T10	Social networks						•							•
T11	**Sales data**	•	•		•			•					•	•

(The list of techniques is based on earlier work; see Table 1. The not described by the earlier work, are marked as **bold**)

Table 7 Sampling techniques identified in the studied cases

Techniques	Cases P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13
Developers as users			•	•		•	•			•	•	•	
Convenience sampling				•		•	•			•	•	•	

The list of techniques is based on earlier work; see Table 1. The techniques not described by the ealier work are marked as **bold**

"We just talk together almost all the time. It is important to establish relationships with them, hoping they become honest. What works for them, what does not work. What is their actual motivation?"

Additionally, the interviews were often used to better understand implicit feedback such as event data **(G2)**.

Interestingly, there were different attitudes in terms of how costly interviews were perceived as a technique. Although some interviews revealed that the interviews were resource consuming and provided little helpful insight (P6, P9), several informants (P1-P4, P10, P13) highlighted this feedback technique as a means to save product costs while acquiring deep insight.

The discrepancy in the attitudes toward user interviews seems to be due to a varying availability of software engineers. While cases with good access to the engineering expertise (e.g., several full-time developers in the team) and hence the ability to experiment regarded the interviews as redundant and costly to implement, the cases with fewer engineers or unstable access to the engineering expertise preferred to save resources by first exploring the user preferences through interviews. In this group, user interviews were therefore perceived as a means to save resources **(G3)**.

In-Product Surveys is a technique that allows evaluating the users' appreciation of the current product through polls or surveys that are built into this product (Fabijan et al. 2015). In-product surveys were mentioned by participants from three cases.

An example of an in-product survey interface is a free text box triggered by the thumb-up or thumb-down icon (P7).

In-product surveys were typically used as a supporting data source to determine the user's reaction to the implemented features (**G5**). For example, in P4, a survey was utilized when the user opted to cancel the subscription, thus giving insight into the reasons for cancelation.

Table 8 Summary of the analysis techniques and the studied cases

Techniques	Cases P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13
A/B testing							•				•	•	
Telemetry dashboards					•	•	•	•	•	•	•	•	
Cross-validating explicit and implicit feedback					•	•	•			•	•		

(The list of techniques is based on earlier work; see Table 1. The techniques not described in the earlier work, are marked as **bold)**

Observations refer to developers observing the users' behavior while interacting with the product (Johanssen et al. 2019) or or without involving the product (e.g., observations of people in their working environment).

Participants in 6 cases reported using observations. The practitioners were either observing the users while interacting with the products (product-related observations) or while performing their daily tasks (ethnography-style observations). Product-related observations could be of two types: nonparticipatory and participatory observations.

Nonparticipatory observations were made in controlled settings (e.g., in an office environment reminiscent of a laboratory) and contained no prototype testing or interview elements. They were applied to understand the user reaction to the implemented features (**G5**). For example, practitioners from P11 reported conducting nonparticipatory observations with video recordings where they invited users to the office to use the product. Such controlled observations were somewhat old-fashioned and time-consuming because the users needed to be available during the office hours and to be recruited at a specific time of the day, which reduced the access of the representative user groups while needing to spend time preparing. The UX specialist commented (P11):

> *"We think it is a bad prioritizing of our resources to recruit a lot of advanced user tests."*

In the participatory observations, the members of the product teams could, in addition to purely observing, ask questions to better understand the user's actions. This made observations somewhat similar to prototype testing (see below). For example, in P8, the UX designers relied primarily on observations during video calls in which users were asked to share their screens to showcase how they interacted with the product.

The ethnography-style observations were reported by P3 and P8. In P8, a subset of the product team observed users for an extended period in their working environment. The team members made approximately five visits over two weeks for a few hours each. The team emphasized that due to the opportunity to spend more time with the users, ethnography-style observations revealed local user differences, provided a better understanding of the specific context of the users (**G1**), and helped establish relationships with the users (**G4**), thus making it easier to collect feedback in the future.

Interestingly, both products where ethnography-style observations were reported, were aimed at niche users (e.g., construction engineers in P3 and public servants in P8). This suggests that ethnography-style observations are beneficial in niche products.

Prototype Testing is the evaluation of partially developed interfaces and functionality in collaboration with users (Fabijan et al. 2015). This may involve both non-functional (e.g., mockups) and functional (low-code) prototypes, depending on the prototypes' maturity.

Prototype testing was mentioned as a preferred user feedback technique by informants in nine products (P1-P5, P7, P9-P11). It was often a combination of sessions with users interacting with the products, participatory observations (see above), and informal user interviews. In other words, practitioners commonly invited the users to interact with the products, observed the interactions, and asked questions to better understand it. In this way, the developers were able to understand the user's reaction to the implemented features (**G5**).

Several product managers (P3, P9) reported asking users to think aloud during user interaction with the product. The result of prototype testing was typically loosely registered qualitative data (e.g., in the form of field notes) or not registered at all, which the product team could use as input to the team discussions and planning sessions, e.g., standups (**G11**).

Insight from prototype testing was considered very helpful in gaining a deep understanding of the user context (**G1**), as indicated by this quote (P1, product manager):

> *"We work very qualitatively, we acquire much more insight through a [prototype testing] meeting where we see what the users do, what they misunderstand, and what they say, rather than just assuming based on some clicks."*

Incident Reports are descriptions of problems generated by users, e.g., support data, trouble tickets) (Johanssen et al. 2019). Although incident reports may not only be user-generated, as suggested by the definition above, many such reports are also automated. However, in the categorization we use in the current paper, the automated reports are instances of event data (see Section 4.1.2) because they produce implicit data.

Incident reports were mentioned by informants from 6 cases (P4, P5, P8, P9, P10, P11).

The incident report analysis aimed to evaluate whether the functionality works as intended (**G6**). As indicated by the following quote, incident reports may be very valuable, as they allow one to detect unexpected bugs and errors early (P10, product manager):

> *"Our pilot customers called us in the test phase and told "hey, there are some things that look very strange". This was very useful in improving the new version of the dashboard."*

In this example, the users directly communicated the incident reports to the product team, which was also reported in P4 and P5. However, in some cases, incident reports were communicated by internal stakeholders who were not users per se, such as colleagues and product sponsors (P8, P10, P11). This highlights the role of the workplace network in the generation of incident reports.

User Requests and Suggestions are descriptions generated by users aiming to express desired features. This type of technique was not explicitly listed earlier (Fabijan et al. 2015), whose list we used as an initial blueprint for the taxonomy of feedback techniques. This techniques is therefore marked with bold in Table 6

User requests and suggestions were brought up as a feedback technique by informants from five cases.

User requests and/or suggestions were described as input initiated by the user, which differs from other techniques (e.g., interviews and incident reports) that product developers initiate. User requests and suggestions may take the form of emails, chat messages, or even oral suggestions made in a personal conversation. We noticed that such feedback was reported when practitioners established more informal relationships with the users, as such relationships may motivate users to actively reach out to offer the feedback. This feedback allowed product developers to reveal sometimes unexpected user preferences (G9).

4.1.2 Implicit User Data Collection Techniques

These are techniques in which users are *not* aware of sharing their feedback with developers for the purpose of changing the product.

The user data collection techniques described by our informants were online ads, beta testing, event data, social networks, and sales data (see Table 6).

Online Ads refers to displaying information about the product at external internet sites to evaluate the user's interest (Fabijan et al. 2015). The resulting user data may reflect the number of clicks, traffic data, etc.

This technique was reported by P2 and P4. For example, in P4, the team created several Instagram posts to determine whether the content related to the product interests users (**G7**) and can attract them to the application. The product manager (P4) commented:

"Very easy to see whether we can create more traffic and trigger the sales"

As seen in this example, online ads may be closely related to another technique called *social networks* (see below).

Beta Testing is an evaluation of an early version of the product with a subset of current users (Fabijan et al. 2015; Yaman et al. 2020). This technique was reported as a data-collection technique by 6 informants (P2, P4, P5, P6, P7, P10).

The difference from prototype testing is that prototypes, e.g., mock-ups or low-code versions, are less mature than the beta-test versions and that beta tests are carried out with a significantly bigger sample of users. For example, P7 launched the early version of Internet banking to about 5000 bank customers to see how this group would respond to the changes. At the same time, beta testing could also be applied to a smaller sample of users. For example, P5 released beta functionality to selected customers to test the beta features. The beta functionality could thus be turned on and off depending on the needs of the product team and the needs of the customer. As the data scientist explained, this approach helps reduce the unfinished code (P5):

"Building the solution in the actual environment gives an advantage of building step by step and at the same time avoiding half-finished code that would have ended up not being used by anyone"

As can be seen in these examples, beta tests were used to acquire detailed insight while reducing the risks of user dissatisfaction thus saving the resources (**G3**). The other reasons were to determine the user reaction to the implemented features (**G5**) and to ensure that the functionality works as expected (**G6**).

Event Data is data collected online per user, e.g., demographic data, behavioral data, and product performance data such as responsiveness, runtime errors, etc. Fabijan et al. (2015). As discussed in subsection 'Incident reports', in current study the event data also includes automated incident reports (in contrast to user-generated).

This technique was one of the most popular and was mentioned by informants from nine cases. This is not surprising given that this technique reflects a wide range of various data sources, theoretically also including those produced with the help of other techniques in this subsection (e.g., online ads, sales data, social networks, etc.). Nevertheless, we have chosen to differentiate this technique from other implicit user data collection techniques because in the majority of cases the origin of the event data was hard to trace.

The general purpose of event data is to provide information on how the user responds to the product (**G5**) and evaluate the product's performance (G6). There was a great variation in the way these data were collected, managed, and analyzed. First, the number of available metrics that were reported ranged from several (mostly in early-stage products) to several dozen metrics in mature products.

Secondly, some practitioners performed additional manipulations to restructure and interpret the data. The event data could be seldom used in its raw form and required other approaches to structure, visualize, analyze, and make sense of the information. In this example a product manager states that evaluating the user's activity with event data requires additional effort (P3):

"We have the change logs in a file. So I used some effort to group these data and create some kind of database. [...] I wanted to know how active each user was when using the product."

As pointed out in this quote, the raw data itself was often insufficient to acquire meaningful insights. Therefore, many practitioners reported additional techniques to interpret and present these data; see Section 4.1.4.

Social Networks refers to the evaluation of the product on social media platforms. Social media activities, such as posts and likes, communicate user frustrations and appreciation about the product to the general public. Hence, they should not be confused with user requests and incident reports targeted directly at developers, regardless of the medium. Social networks were described as a feedback source in P13.

Online ads and social networks can be closely related, and thus difficult to differentiate, since online ads are often placed on social media platforms such as Instagram and Facebook. For instance, P13 used social networks as a proxy for the products' popularity, e.g., the number of likes per picture, to evaluate how much the users appreciated each architect's design. The goal of such an evaluation was therefore to determine the user's interest in the product (**G7**).

Sales Data refers to implicit data characterizing a product's potential or current popularity among users and profitability.

Sales data is an example of a user data collection technique that was not listed by Fabijan et al. (2015) and is thus marked in bold (Table 6).

Informants from five cases (P1, P2, P4, P7, P13) reported collecting data on product sales. Examples of such data were the number of users who subscribed to an information letter (P2), the number of users willing to buy subscriptions (P2, P4), and the number of visitors to the product's Google Play page (P12). Actual product sales reports were also mentioned (P7, P13).

The purpose of collecting sales data is to assess profitability, but not in all cases. Although in some cases the goal was indeed assessment of the product's ability to generate revenue (**G10**), in others these data were collected to evaluate the product's popularity (**G7**).

An illustration of the usefulness of sales data comes from cases P2 and P4. The two teams jointly developed a solution that assessed the willingness to pay among potential customers. This Figma-based prototype was identical to the actual application, but did not contain real code. By offering users various subscription plans, the prototype allowed counting the number of users willing to subscribe to each plan (free, priced subscription, or one-time purchase). This provided insight into how the products' pricing affected the interest of the users.

Although sales variables do not appear to be related to software development, they were also described as important to engineers. This is because functionality, even at an early product stage, should be adapted to the mechanisms that trigger sales or scaling (gradually increasing number of users).

4.1.3 User Sampling as Feedback Techniques

Sampling was not initially the focus of this study, which was mainly exploring techniques to collect user input after users were sampled. However, strikingly many informants described sampling as a way to acquire user feedback, which is why we chose to include this category of feedback techniques in our report. These techniques did not have to do with how the

feedback was elicited but rather were determined by the way the test users were sampled, e.g., for interviews, prototype testing, etc. We summarize our findings on sampling techniques in Table 7. The dots in the table indicate the cases where each technique was identified.

Developers as Users This source of user feedback implies that the members of the product teams act as test users of their own products (Fabijan et al. 2015).

This technique was reported in surprisingly many cases (P3, P4, P6, P7, P10, P11, P12) and was primarily described as beneficial and easy to use because it allowed quick and cost-effective feedback elicitation.

Functioning as user proxies allowed such team members to see the product from the user's perspective (**G1**), understand how a user might perceive the functionality (**G5**), and whether it works as intended (**G6**). Overall, "Developers as users" was perceived as very useful and cost-efficient. This allowed saving resources that would have been spent recruiting and conducting tests with users (**G3**). The benefits of this technique were even more pronounced when the team members were able to make the necessary adjustments themselves, thus also reducing the wasted time. The technical lead of P7 explained:

> *"We, developers, can quickly discover a bug while we use the mobile bank: Oh, here is a bug, I can fix it tomorrow."*

When using developers-as-users, a domain experience relating to their product was described as especially beneficial among developers (P3, P4, P6). This domain experience allowed engineers to better understand how a potential user might experience the interface, as explained by a product manager (P3):

> *"I am an engineer myself and have been working on such projects, so I had an idea of how I want the tool to be."*

Convenience Sampling is a user sampling approach when developers recruit users from their closest network, for instance, family members, friends, colleagues, etc. This approach is similar to "Developers as users" in that the user sampling is not random, which can introduce biases in the user response, e.g., parents and friends giving positive feedback due to social desirability. This technique was not listed as a feedback technique in the list of techniques we used as the initial blueprint (Fabijan et al. 2015), which is why it is highlighted in bold (see Table 7).

In particular, this approach was surprisingly common, as it was mentioned by six informants (P4, P6, P7, P10, P11, P12).

Convenience sampling was described as a way to access feedback quickly and cost-effectively. Thus, this technique was used to understand the user context (G1) and to save resources and time (G3). For example, developer in the P11 case the developer that demonstrated new features to his fourteen-year-old daughter, stated:

> *"It was just to get a hunch whether this [new function] is off the hinges or is understandable."*

After such low-effort testing that helped identify obvious drawbacks and errors (G6), a team would typically release the updated code to end-users while closely monitoring their behavior by collecting event data.

The precondition of such monitoring is the possibility of accessing usage data in a way that is structured and easy to interpret, e.g., in the form of a telemetry dashboard (see the respective subsection). Such a dashboard applied after the initial release would reveal the

degree and character of adoption of new features. In such cases, the product team would investigate and consider rolling back or changing the functionalities.

4.1.4 Feedback Analysis and Presentation Techniques

A subset of the reported techniques involved product developers making sense of the user feedback (e.g., analyzing, visualizing, and discussing with others, etc.). We have chosen to describe these techniques in a separate category. The techniques in this category are A/B testing (Fabijan et al. 2015, 2017), telemetry dashboards, and cross-validation of explicit and implicit feedback. We summarize our findings in Table 8.

A/B Testing means displaying different versions of the same products to evaluate the version producing the desired response (Fabijan et al. 2015; Yaman et al. 2020). This technique allows choosing the best design solution between two competing options by displaying both designs (A and B conditions) to independent user groups. We classify this technique as an analysis technique because its essence is in statistical comparison (analysis) of even data from two conditions and not a collection of user data. Cases P7, P11, and P12 brought up this technique.

Although A/B testing is sometimes considered a data collection technique (Fabijan et al. 2015), its purpose is to analyze the data collected by other means, e.g., through event logs. Therefore, we chose to mark A/B testing as a data analysis and representation technique, not a data collection technique.

The A/B tests were preferred when the product teams needed minor adjustments to existing features and therefore found the condition with the highest user engagement (**G8**). A precondition for this type of feedback was a sufficiently high number of existing users, providing significant differences between the A and B conditions. This explains why this technique was not applied in early-stage products.

An advantage of A/B tests was relatively unbiased results due to a high number of user observations and the possibility of statistical analysis.

Another advantage was that the results could be easily communicated to other stakeholders. A vivid example of this is P7. When the product team changed the entry text to one of the features in their online banking application because the A/B test showed that one version triggered additional sales. The example also shows the benefit of cross-validating the output of the A/B test with sales data, as pointed out in this quote (P7, tech lead):

> *"We could easily calculate the business value by how many more customers ordered the product. It was a great success!"*

This success case, however, can be an exception as the same informant added that many other A/B tests were not equally easy to interpret. This drawback may also explain why more informants did not report A/B tests.

Telemetry Dashboards is the infrastructure that helps summarize, visualize, and analyze the implicit user data (e.g., event logs, incident reports, etc.), which product developers use for continuous monitoring and planning of the products' evolution.

Whereas some teams were collecting the implicit user data without necessarily aggregating it and comparing it to other data sources (P3), the majority applied additional tools (dashboards) to keep the event data in one place, visualize and follow it up (P5, P6, P7, P8, P9, P10, P11, P12).

Telemetry dashboards allowed one to achieve multiple goals, such as determining the user's reaction to the implemented features (**G5**), the product's popularity (**G7**), and reliability

(**G6**). Furthermore, such tools could be used effectively for product planning (**G11**). For example, the product manager in P5 told us that the team continuously used the dashboard to make decisions on which features should be prioritized and which should be eliminated, as described below:

> *"We have had the metrics from day one [..] And we have a metric for all the functionality and how it is used. If we see that some numbers are not giving much insight, we can remove some metrics and add others. The dashboard can't be static when the product, the business, and the customer change all the time."*

The ways of building and maintaining dashboards were different. For example, while several teams described using standard solutions, for example, Amplitude in P12 and Mixpanel in the case of P6, some developed the dashboards themselves (P5, P9).

Relying on an off-the-shelf solution was beneficial to saving development costs for early-stage products, but could be limiting in the long run. For example, the product manager of P6 told us that the team was enjoying a discount subscription at Mixpanel that is available for startups but that it could be problematic and costly to lose the current plan in the future.

In contrast, self-developed dashboards were described as more beneficial because they allowed data to be pulled from different sources and combined with product event data (e.g., behavior data, product data, and incident reports). In addition, self-developed dashboards could be continuously adjusted according to the data which prove useful for product monitoring and planning.

At the same time, developing and maintaining such dashboards from scratch was considered too costly, especially for early stage products (P1-P4), because it required specific expertise and development capacity. Additionally, the quality of the event data for such products may be low due to the small number of users.

However, the need to visualize, structure, and interpret event data was high in all cases. Even early stage products without access to event data were often concerned with structuring and meaningfully following up the user data. They were looking for alternatives for the dashboards that allowed them to monitor the metrics without needing full-scale development or purchase. For example, the P2 team manually monitored the changes in their user metrics using a conventional Excel spreadsheet. As the product manager stated:

> *"Dashboards are great for following up the product development."*

Cross-Validating Explicit and Implicit Feedback is an approach for interpreting user insight by comparing the explicit and implicit sources of feedback. In this way, practitioners may understand various data sources and put them into perspective. This technique was identified in four cases.

Using telemetry alone may not be sufficient to understand user behavior as the reasons for such behavior may be unclear. To address this issue, several product teams cross-validated the implicit insight from the telemetry with the explicit user feedback acquired through interviews and prototype testing. The owner of the P9 product explained that doing so is important for deep understanding behind the users' intentions (**G1**):

> *"When we can ask them what they [the users] were thinking, we often get eureka moments."*

The P5 case provides an interesting example of how cross-validation can be performed. The team developed a dashboard for monitoring user telemetry at a very early stage of product development. Subsequently, the quantitative dashboard was used during user interviews to help the users reflect on the product's functionality and usefulness, thus facilitating their

qualitative feedback. In this way, developers could assess the user reaction to the implemented features (**G5**).

Table 5 summarizes the identified goals for collecting user feedback for each feedback technique. The analysis of the goals was performed to achieve a better understanding of each feedback technique. However, we chose to include an overview of the goals to give a better understanding of possible relations between the goals and the techniques.

The most common goals were **G5:** determine the user's reaction to implemented features, **G6**: determine if functionality works as expected, **G1**: understand the user's context, **G7**: determine user's interest. The least reported goals were **G9**: reveal user's preferences and **G10**: determine user's willingness to pay.

The table suggests that diverse user feedback techniques can be motivated by the same goals. For instance, interviews, observations, prototype testing, convenience sampling, developers as users, and cross-validating feedback (implicit and explicit) - all may be utilized for understanding the user and their context. This suggests that the likelihood of achieving each goal increases with feedback redundancy.

At the same time, each feedback technique can contribute to multiple goals. For example, telemetry dashboards help determine user reaction (**G5**), whether the functionality works correctly (**G6**), whether the user is interested in the overall value proposition (**G7**), and help plan the development process (**G11**). Such examples indicate that a high number of the related goals may justify the implementation costs of a particular technique.

4.2 Challenges Related to User Feedback in CSE

Our data analysis resulted in six groups of challenges, which are summarized in Table 9. The challenges reported the most frequently were related to resources and metrics. We will now describe each group of challenges.

4.2.1 User Problem-Related Challenges

This group of challenges was typical for products where the user problem specification was uncertain for various reasons, such as new products or several user groups. The user problem is the understanding of the problem the user solves with the product and how the product contributes to solving this problem (Ros et al. 2024).

When the user problem was unclear, it was often problematic to acquire user insight because the user group itself was not sufficiently specified. For example, the product team of P2 was struggling to recruit test users because the user group itself was not yet well understood. The product manager complained:

> *"We need to understand how to recruit the test users, which tone of voice we will use, which pitch we use."*

In the B2B settings (P1 and P10), the challenge was to find a user problem that would be generic enough to be suitable for several corporate customers and not tailored for one particular company. The product manager of P10 explained:

> *"We need to rely on particular cases to solve the generic problems, the product cannot be tailored to each and every customer."*

To address this challenge, practitioners reported increased interaction with potential users to better understand the possible needs and how software products can address them, e.g.,

Table 9 Summary of the identified challenges and the studied cases

Category & Description	Cases												
	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13
User problem - Uncertain user problem or user group	•	•						•		•			
Data acquisition - Insufficient access to user data, awareness of user data within one's own organization								•		•	•		
Metrics - Hard to choose and maintain meaningful metrics for monitoring user behaviour and product success; Difficult to interpret metrics			•		•				•		•	•	
Resources - Insufficient time and/or funding		•	•	•		•		•	•	•			•
User satisfaction - Users' negative reaction to changing interfaces							•						
Method - Methodological shortcuts in capturing user feedback						•			•			•	

through interviews and observations. For example, ethnography-style observations were described as valuable for understanding the user problem, especially with respect to niche users (see Section 4.1.1).

Another aspect of a poorly specified user problem was the reduced ability to evaluate the product's potential profitability, which some informants reported. Having a limited number of pilot users at an early stage (as in P1) prevented the product team from evaluating whether the business model could be scaled to a larger pool of customers. In other products (P2, P10), the challenge was similar: How to attract a sufficient number of users for testing when the product is not mature enough to provide the functionality? The product manager of P2 explained:

> *"We know that we cannot test people's willingness to pay because we do not yet know how the app will work."*

As a remedy, the product managers described collecting sales data (see Subsection 4.1.2. In this way, practitioners could iterate on ways to communicate with the users that capture the most user interest, thus indicating the actual user problem.

4.2.2 Challenges Related to Data acquisition and Data Sharing

Various user data sources are often sources of ideas for products (e.g., event data) or the foundation of the product's functionality (such as dashboards and other analytics visualization tools). In this category, we grouped the challenges related to acquiring such data, as indicated by informants from three products.

User Data Privacy Complying with privacy regulations when working with user data, was described as challenging in two cases P8 and P9. Specifically, what user data were legal to collect, share, and store, was often unclear. Both cases belonged to the public sector and thus worked with real citizens' data that were large and complex. Because of this, work in these products was highly data-intensive

Firstly, the societal responsibility of ensuring that citizens' data were kept safe had become a heavy burden in these cases. Secondly, it was challenging to apply data regulations to understand whether user data can be shared, as the regulations were not sufficiently context specific. This prevented data sharing and collaboration within organizations and with other actors.

The hesitation of sharing user data eventually could lead to poor data quality. For instance, for P9 and P11 this resulted in a "better safe than sorry" mentality, entailing that the product developers settled for lower data quality to lower the risk of defying the regulations. In expressing his fear of accidentally mishandling data that could be deemed private, one developer explained:

> *"I am terrified of a newspaper headline showing an individual's sensitive data being leaked. That would be a disaster."*

A suggested remedy was related to expanding the competence of a product team. In case P8, a legal expert was appointed to work on a regular basis. This helped developers by supporting technical decisions related to data privacy. Eventually, the presence of a legal expert led to a drastically shorter decision time related to data management.

Data Awareness Knowing about the existence of data sources internally was also problematic, especially in big companies where data awareness depends on one's social network.

For example, being a new hire or a consultant with a small internal network could prevent product managers from knowing the right people to ask for data, or even knowing which data is available. As pointed out by a UX designer of P11:

> *"We can ask the data team through a developer in our team […] but then we need to know what to ask for and whom to ask."*

4.2.3 Metric-Related Challenges

This set of challenges is related to quantitative metrics (telemetry) used for product monitoring (e.g., event logs, number of downloads, conversion rate, etc.).

Choosing and Prioritizing the Metrics Continuously was described as challenging in new products where the user problem was uncertain. Because if this, it was unclear what kind of user behavior should be monitored. For example, the challenge of continuously updating the metrics was described in one case (P5). Although the product team established several metrics at the early product stage, there was a need to continuously update the metrics to ensure their meaningfulness. The product manager of P5 confessed:

> *"The challenge with being data-driven is to choose the right metrics. Because one can measure so many different things."*

One solution used by the P5 team was to continuously monitor and discuss the metrics (e.g., during the daily standups), which helped provide some foundation for choosing the metrics.

Another metric-related challenge was related to the dynamic character of the metrics. In P9, metrics were described as unstable as typically stopped working as intended a couple of weeks after their creation. This occurred due to numerous changes to the product and the user interface. The problem led to additional costs spent for recovery of the metrics, as described by the product owner of P9:

> *"When a metric break down you have to use a developer, which costs 12,000 crowns (approximately 1200 euros) a day to fix it and keep it up to date."*

The informants did not provide a solution to this challenge. The developers in case P9 eventually chose to let the measurements gradually "die out" to save the recovery costs. They accepted that metrics have a very short time span and continuously assessed whether the corrupted metrics provided user feedback of sufficient value to invest additional resources in maintaining them.

Interpreting the Metrics Several product teams (P12, P9) described that measuring the users' reactions via proxies was hard to interpret. For example, the product manager of P12 had access to a large amount of user data with various metrics but admitted that these data were unable to explain the user behavior, as indicated by this quote (product manager, P12):

> *"It is a lot of data. So it is like you see only shadows and need to make sense of them"*

Since interpreting the metrics was challenging, the risk of misinterpretation was high. The informants from another case (P11) gave a prominent example of how metrics can be misinterpreted. The team measured the total number of app downloads from iTunes and Google Play to determine the effect of a marketing campaign. A rise in download numbers showed

that the campaign was working; however, the developers later found that this conclusion was based on incorrect assumptions, as a bot probably downloaded the application. The developer commented:

"Only a few years ago [..] you could assume that downloads meant that people used an app; it's not like that anymore."

Since interpreting user behavior was hard, planning the product was also described as challenging. Despite having access to a large amount of implicit user data, creating product-related hypotheses and prioritizing backlog items was still not a straightforward task.

As a remedy, the practitioners suggested several ways to analyze the data from the event logs. The most popular approach was interpretation with the help of telemetry dashboards (as described in the corresponding subsection).

Other approaches were also reported. For example, the tech lead of P7 applied explorative statistical analysis to find patterns that can be used for product planning.

In addition, several informants (P1, P2, P5, P8) reported team discussions as a means to make sense of both quantitative and qualitative user data. Such discussions often took place during the team's stand-ups and had the goal of extracting backlog items.

Additionally, some informants suggested that the interpretability of implicit user data can be improved by cross-validating it with explicit user feedback, specifically the data captured through user interviews (see subsection "Cross-validating implicit and explicit feedback").

Finally, growth analysis was reported as a means to predict product profitability based on current popularity of the product and its conversion rate on Google Play (P12).

4.2.4 Resources-Related Challenges

These challenges were related to the lack of resources available to product teams, such as financial or human resources. The informants from eight cases, see Table 9, pointed out that the shortage of resources resulted in the need to prioritize the data collection techniques that required the least time and software development capacity. The usual consequence of that was a reduced ability to collect and monitor usage data (e.g., event logs), as it often required the participation of software engineers to collect and represent this type of data systematically. For example, the product manager of P4 admitted that the team had only limited implicit feedback because most of the available work hours were used to follow-up with pilot customers. This was an important challenge in understanding how to best sell and promote the product. The product manager said (P4):

"It would be very nice to have all the numbers on how the users interact with the service somewhere on Amplitude, but this is not something that we have today"

The lack of human resources (such as product developers) resulted in higher preferences for explicit user feedback techniques. For example, in three cases (P2, P3, P11), the shortage of resources led product teams to pay less attention to product monitoring in favor of interviews and prototype testing. Similarly, reduced access to software developers in the case of P10 slowed early stage development, preventing the product manager from creating and improving prototypes based on user feedback. The product manager commented (P10):

"If the IT had been there from the start asking relevant questions [..] I would know what to deliver."

Interestingly, in cases with good access to software developers (P6, P9, P12), the need to save resources resulted in an opposite approach. For example, in P6, the team relied on

the up-front deployment of features with subsequent validation of the actions by telemetry. The product manager explained that they did it in such a way because acquiring user insight upfront (such as user interviews and prototype testing) could be too time consuming. They said:

> *"We are really limited in terms of work hours, so we did not run user tests and so on; we just agreed on what gives the most value for the users and then sent it to production."*

Such a lean approach allowed the team to save time developing the functionality quickly and further iterating based on the explicit feedback. For this particular team, this was a good solution because the team consisted of three software developers who were able to quickly create and change code.

4.2.5 Challenges Related to User Satisfaction

This set of challenges was related to the user's negative response to the changes caused by experimentation with the user interface.

Attempts to offer a better user experience could cause both positive and negative reactions. In one case (P7), A/B testing a new feature (view of the dashboard in the net banking application) led to negative feedback from users, measured by short online surveys. Although most of the beta users liked the new setup, numerous users were unhappy after the feature was released for the core customers. However, the product team convinced the management to keep the new setup, as it brought more business value. The team hoped that the users would gradually habituate, which eventually happened. The tech lead from the product team emphasized:

> *"We see that the satisfaction went up, so after some time, the users were equally happy as they were before."*

This example shows that techniques such as A/B testing may lead to paradoxes because attempts to improve user experience can also threaten users' satisfaction. In particular, the example provides a success story, as user satisfaction eventually increased, thus justifying the implemented change and the captured feedback.

4.2.6 Method-related Challenges

These challenges were linked to dissatisfaction that product developers expressed toward the degree of rigor in their work with user feedback. Specifically, the issues of not collecting sufficient user feedback and not analyzing the feedback were reported. Since analyzing a large amount of user feedback is associated with significant effort, some informants acknowledged that they were looking for user feedback with high interpretability. For example, in the case P9, the choice of telemetry and its subsequent analysis was described as insufficient. The owner of the product of P9 stated:

> *"We just set up some metrics, see what users click on. And then we can tick the user insight box as done."*

Such "cherry-picking" of data points resulted in focusing on only a subset of feedback while ignoring the rest.

Notably, people who reported insufficient rigor were often aware that they were doing something wrong. However, they were doing it anyway. Although in some cases, the lack

of rigor was motivated by the need to save resources, see Section 4.2.4, in other cases, the motivation was unclear.

5 Discussion

We have described user feedback techniques, the purpose of applying them, and the ways to present and analyze feedback as reported in 13 industrial cases relying on CSE. Based on these findings and on our previous work (Klotins et al. 2022; Klotins and Gorschek 2022), we have summarized the key terms from our results and mapped them to the model of CSE; see Fig. 2. In the figure, we depict an earlier conceptual model of CSE (light gray) and concepts added by this study (dark gray). The overlap concepts are highlighted with dark text. The dashed lines denote associations between the concepts. As shown in the figure, the process of collecting and applying user feedback is related to the bottom part of the CSE model, specifically with continuous use, monitoring, and product planning.

The process can be described through the concepts that we described in our results: feedback techniques, analysis and presentation techniques, and data collection goals. In the next subsection, we continue to discuss our results by detailing each concept and how they correspond to the CSE process, thus answering research question 1: *How is user feedback collected and applied in CSE?*

5.1 RQ1: How Is User Feedback Collected and Applied in CSE?

The general process of collecting and applying feedback in CSE can be described in the following way (see Fig. 2). *Product developers* apply *feedback techniques* based on which *product goal* needs to be achieved. The application of feedback techniques is possible due to the continuous use (Bosch et al. 2018; Klotins et al. 2022) when end users access the product over time (as opposed to initial adoption). Further, the *user feedback* that is captured through feedback techniques must be interpreted by applying *analysis techniques*. This allows

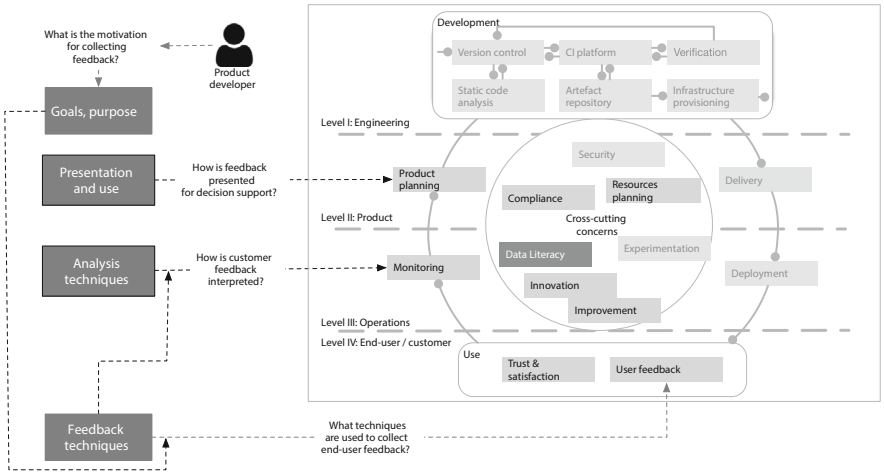

Fig. 2 The overview of our results and their mapping to earlier results

feedback to be used in continuous monitoring (Bosch et al. 2018; Klotins et al. 2022). The feedback can also be used to inform *continuous product planning* (Bosch et al. 2018; Klotins et al. 2022), which is made possible by *feedback presentation and use*.

We will now discuss each concept with examples from our results in relation to CSE. The main findings of this subsection and their relation to previous literature are summarized in Table 10.

Feedback Techniques are used to elicit implicit and collect explicit end-user feedback. User feedback refers to all feedback and data collected from (and about) current and potential users. The selection of feedback collection techniques determines what kind of feedback is collected and how it can be used for further analysis and representation.

We found that the feedback techniques in CSE are largely the same as those in software engineering in general. From the list of techniques reported by Fabijan et al. (2015) in 2015, the majority were also reported in this study. The exceptions were theater sessions, walk-throughs, and crowd-funding platforms, which we did not identify in our sample (possibly because the sample was relatively small).

In addition to the feedback techniques in the initial list, our informants also reported two distinct techniques: user requests and sales data.

The application of user requests implies that users provide descriptions of desired features or changes. This technique highlights the role of the users' initiative when collecting user feedback, since this feedback tends to occur without encouragement from product developers. The value of such feedback may be high since the feedback on one's own initiative is likely to reflect an honest opinion. Additionally, such feedback may imply that the user is satisfied with the product and wishes to continue using it. Otherwise, requesting new features would be meaningless. This seems to be related to the concept of continuous trust, which means that the user accesses the product over time based on their trust in the quality and security of the product (Bosch et al. 2018; Klotins et al. 2022). Therefore, this feedback technique may be especially important in CSE. Sales data as a feedback technique refers to using data that have to do with the product's potential or current popularity among users and/or profitability (such as number of downloads, purchase clicks, etc.). Although sales metrics per se do not seem immediately relevant for software engineering, we have learned that this feedback is important to develop features that increase the popularity of the product.

Further, we found that some techniques were determined by the way the test users were sampled: convenience sampling and developers as users.

Convenience sampling means that the test users are being recruited from the personal network of the product developers (e.g., friends, relatives, etc.). This technique allows for collecting feedback in a cost- and time-efficient manner, and was due to this being unsurprisingly popular among the informants. Although this technique has apparent disadvantages (high risk for biased feedback due to non-randomized user sampling), it was described as very valuable because of high availability and interpretability of feedback.

Developers as users implies that product developers perform an internal evaluation of their own product (Fabijan et al. 2015). In essence, it means that the developers sample test users, which makes this technique an instance of convenience sampling (see above). This technique was also reported often and was described as pragmatic and effortless. This is somewhat surprising because earlier this technique was described as time consuming (Fabijan et al. 2015).

The difference between our results and earlier findings may be explained by the context in our products. Both convenience sampling and developers as users were primarily reported

Table 10 Summary of the state of practice related to user feedback in CSE

Element of the conceptual model (Fig. 2)	Key findings	Related literature and concepts
Feedback techniques	The feedback techniques in CSE are largely the same as those in software engineering in general	Fabijan et al. (2015)
Feedback techniques	Additional feedback techniques are identified: user requests and sales data	Continuous trust, continuous use Klotins et al. (2022); Fitzgerald and Stol (2017)
Feedback techniques	Techniques determined by the way the test users were sampled (convenience sampling and developers as users) were surprisingly popular	Fabijan et al. (2015), user problem (Ros et al. 2024)
Analysis and presentation techniques	Differentiating analysis and presentation techniques from feedback capture techniques is important to conceptualize user feedback within CSE	Fabijan et al. (2015); Bajic and Lyons (2011)
Analysis and presentation techniques	Telemetry dashboards were often highlighted as a way to analyze implicit feedback	Continuous monitoring, continuous planning (Klotins et al. 2022)
Goal (of a product developer)	Multiple goals for collecting user feedback are identified. The goals moderate the selection of feedback collection techniques. A new finding is that some feedback can be collected to validate other feedback sources	Johanssen et al. (2019)
Goal (of a product developer)	Many reported goals, perhaps not surprisingly, were related to understanding the users and their reactions to the software	User problem, product-market fit (Ros et al. 2024)

by the same cases, indicating that the product context (e.g., culture, practices, user type) may influence whether such sampling techniques are preferred or not. We can assume that such lightweight sampling techniques are especially beneficial in products that are meant for the general user because the perception of such users may be similar to the perception of any user (including the friends or relatives of a product developer). However, our sample was too small to confirm or reject this assumption.

The state of the art reveal that the reliance on already known sources, using purposive and convenience sampling, is widely used among researchers. Such techniques are prone to bias. For example, purposive sampling is inherently subjective. Convenience sampling studies subjects that are close and easy to reach; therefore, the results obtained using this technique may be difficult to generalize to a broader population (Baltes and Ralph 2022).

The product context can also account for which feedback techniques are preferred. For example, informants from niche products described the techniques that allow one to capture user context (such as ethnography-style observations) as more beneficial. Again, this may be explained by the complexity of the user problem (Ros et al. 2024), such that users in niche domains may have more complex user problems because their domains differ significantly from others. For example, Sporsem et al. (2023) found that ethnography-style observations were essential to understand niche users in the maritime domain. Based on these and our findings, we can expect that in niche products explicit user feedback techniques such as ethnography-style observations should be used more often.

Analysis and Presentation Techniques are used to make sense of the collected feedback with its subsequent presentation and use in product decision support. We found that some techniques that were earlier described as the ways to collect user feedback (e.g., A/B testing) (Fabijan et al. 2015) are actually a way to analyze the user data such as event logs and telemetry. In the same way, several ways to interpret the user feedback, such as telemetry dashboards and cross-validation of explicit and implicit feedback, were reported.

In our results, analysis and presentation techniques are summarized in the same category. However, it is logical to differentiate analysis from presentation and use because the two contribute to different steps of CSE. Therefore, Fig. 2 outlines "Analysis techniques" and "Presentation and use" as distinct concepts.

Analysis techniques allow to make sense of user feedback, thus contributing to the concept of continuous monitoring (see Fig. 2. This practice helps to evaluate how the product performs in a live environment (Klotins et al. 2022). However, such evaluation is impossible without at least basic analysis (such as metric selection and visualization).

Our results show that differentiating analysis and presentation techniques from feedback capture techniques is important to conceptualize user feedback within CSE. The literature on user feedback (Fabijan et al. 2015; Bajic and Lyons 2011) tends to focus on feedback collection techniques and less on analysis and presentation. At the same time, interpretation can be the cornerstone of feedback work in CSE. In fact, in the situation where the amount of user data is increasingly high due to automation, the ability to interpret the feedback is decreasing. This is especially relevant for cross-validation of explicit and implicit feedback, because it cannot be automated in the same way as A/B testing. We will come back to this issue as we continue to discuss our results.

Our informants pinpointed the telemetry dashboards as a way to analyze implicit feedback. Such tools were described as extremely valuable both by those who successfully applied them and those who only planned to do so. The use of the telemetry dashboard in the online

rehabilitation solution (P5) can serve as a success case when the team continuously monitored the tool to make prioritization decisions.

These findings indicate that telemetry dashboards can contribute to several steps in CSE. Its data analysis potential allows it to be used for continuous monitoring (see Fig. 2). The possibility of presenting large amounts of implicit data helps in continuous product planning. More importantly, as shown by the case of the rehabilitation platform (P5), dashboards can also be adjusted throughout the course of product development, thus ensuring that the visualized metrics facilitate product improvement at every stage. Therefore, telemetry dashboards as an analysis and presentation technique seem to have a significant potential to enhance the CSE process. For the continuation of this discussion, see Section 5.3.

Goals of a product developer determine the need for collecting user feedback in a specific way. The goals moderate the selection of feedback collection techniques.

This study has identified a multitude of goals for collecting user feedback. The goals were related to understanding the users' preferences, quality assurance, reducing the process and the resource waste, interpreting the feedback, and to product planning. Here, our findings are similar to those of Johanssen et al. (2019), who found that user feedback is used for bug fixing, decision support, and product planning. A new finding is that some feedback can be collected to validate other feedback sources. This highlights that feedback interpretation and analysis require not only additional effort, but also, yet additional feedback.

Many reported goals, perhaps not surprisingly, were related to understanding the users and their reactions to the software (G1, G5, G9). This understanding is often required to solve a user problem, that is, to determine what problem the user solves with the product and how exactly the product contributes to solving this problem (Ros et al. 2024). Achieving such a goal was often facilitated by collecting explicit user feedback (e.g., interviews, prototype testing, observations). This suggests that solving a user problem requires deep insight provided by explicit feedback techniques.

Another group of identified goals related to estimating how many users are potentially interested in the product (G7, G10). This understanding may be required to achieve the product-market fit. The product-market fit is a measure of the product's ability to satisfy the market and generate economic value (Ros et al. 2024). Measurement of product-market fit can be made possible by user surveys and sales data (Ros et al. 2024). This is exactly what we found in our study. Informants who reported the goals related to user interest described implicit feedback techniques (such as online ads, social networks, sales data) as means to achieve these goals.

Based on these arguments, we can expect that explicit user feedback techniques fit well to solve the user problem, whereas implicit user data collection techniques are better suited to explore the product-market fit. We suggest that this assumption be validated with a larger case sample.

5.2 RQ2: What are the Challenges Related to the use of User Feedback in CSE?

Our informants reported challenges related to user problems, data acquisition, metrics, resources, user satisfaction, and the method of working with user feedback. By relating these challenges to the CSE model, we will now answer research question 2: *What are the challenges related to using user feedback in CSE?* The list of challenges and their relations to the CSE model and our results are summarized in Table 11.

Table 11 Summary of the challenges in CSE

	Challenges	Relevant element(s) of the CSE model (Fig. 2)	Challenge label in the results (Section 4.2)
1	Continuous user feedback may compromise continuous trust	Trust & satisfaction	User problem
2	Continuous feedback requires a trade-off between feedback quality and the cost of feedback acquisition	Resource planning, Monitoring, Product planning	Resources
3	Continuous compliance hinders the access to user feedback	Compliance	Data acquisition
4	Continuous monitoring requires continuous validation of the key metrics	Monitoring	Metrics
5	Insufficient data literacy may reduce the value of continuous feedback	Data literacy	Method

5.2.1 Continuous User Feedback May Compromise Continuous Trust

We found that one consequence of the systematic capturing of user feedback can be lower user satisfaction (user-related challenge). This is due to incremental changes that are sometimes introduced to capture the user's reaction (e.g. during A/B testing). Earlier findings also suggest that techniques such as online ads and product surveys can be perceived by customers as disturbing (Fabijan et al. 2015) and that, especially in CSE, users dislike frequent changes (Khomh et al. 2015). If users are unhappy with the changes in the product or the need to relate to these changes, they may be less willing to continue using the product.

The model suggests that continuous use is a necessary component of CSE; see Fig. 1. Continuous use implies that the end user uses a product over time, in contrast to initial adoption (Fitzgerald and Stol 2017). Without continuous use, continuous feedback is impossible.

At the same time, a precondition for continuous use is continuous trust, which is the user's expectation that the vendor will act cooperatively and without exploiting the user's vulnerabilities (Fitzgerald and Stol 2017). If the user is dissatisfied with their experience of the product due to continuous experimentation and feedback capturing, continuous trust is at stake. However, such dissatisfaction can again turn out as low feedback quality (Ros et al. 2024).

5.2.2 Continuous Feedback Requires a Trade-off Between Feedback Quality and the Cost of Feedback Acquisition

Our findings suggest that product developers often lack enough time and resources to work with user feedback (resource-related challenges). In this situation, many of them opt for low-hanging fruits rather than not having any feedback at all. For example, low-cost sampling techniques, such as user-based development and convenience sampling, allow the collection of user feedback in a short time with limited time and investment. However, the generalizability of this feedback can be questioned. This is similar to our earlier findings that to increase the pace of software delivery in CSE, practitioners may sacrifice other aspects of the development process (Klotins and Gorschek 2022).

Resource-related issues are not specific to CSE; however, they may have specific consequences in this context. One reason for such issues may be resource planning, which is a cross-cutting concern in CSE (see Fig. 2). Our earlier work shows that resource allocation in CSE is often periodic rather than continuous (Klotins et al. 2022). This may cause inflexibility, and hence instability when it comes to resources.

In addition to financial resources, time seems to be an even bigger concern in CSE. The ever-increasing release speed and amount of automated user data may create time pressure for product developers to analyze the feedback and send new features as soon as possible, forcing them to prioritize feedback acquisition over its quality. Therefore, this trade-off may be an inherent challenge in CSE.

In their efforts to save resources, practitioners in our sample reported prioritizing feedback techniques that could save time or costs. Interestingly, which techniques these were, depended largely on the products' content. Some practitioners relied more on explicit upfront feedback (such as interviews and prototype testing), whereas others preferred implicit post-deployment feedback.

Explicit feedback techniques (e.g., user interviews and surveys) are known to be time consuming (Fabijan et al. 2015). However, we learned that such techniques applied pre-development allow one to save the cost of development, which might be suitable for products

where access to software engineers is limited. In contrast, explicit techniques applied post-deployment allow assessing the users' reactions to the implemented changes in functionality, thus saving time in products where developers can ship the changes fast. Therefore, what is time-efficient depends on the context of a particular product.

Although practitioners in all cases reported collecting both implicit and explicit feedback, cross-validation of the two types of feedback was reported only by a subset of informants. Earlier literature indicates that the problem is typical for software engineering in general and for CSE in particular. Practitioners tend to prefer either implicit feedback (Johanssen et al. 2019; Yaman et al. 2020) that offers generalizability; or explicit feedback (Maalej et al. 2015; Bajic and Lyons 2011) that provides deep understanding. However, relying on only one feedback type can be problematic. When relying on explicit feedback, developers can uncover the deep reason behind the users' behavior, but at the same time have to trust the users' account, which may not always be reliable.

Looking at this problem through the lens of our earlier results, we see its disruptive potential for CSE. Without adequate feedback interpretation and analysis, continuous monitoring and planning (Klotins et al. 2022), which are among the critical activities of CSE, are disconnected from continuous user feedback. In other words, the actual development tasks are no longer based on user feedback, although the feedback is being collected. Therefore, capturing user feedback isolated from its analysis makes monitoring and product planning pointless and even wasteful. What is the use of numerous data points if they do not contribute to the actual backlogs and code?

Of course, the complete absence of feedback analysis is extreme, which we did not observe in our cases. However, there were tendencies to cherrypick when only the feedback that is simplest to interpret or makes the most sense was analyzed. Apparently, such a fragmented approach puts the quality of feedback at risk and thus also represents a threat to CSE.

In summary, the constant need to prioritize techniques that are time- and/or resource-efficient may threaten the quality of feedback. This trade-off, in turn, is disruptive for CSE, as feedback can ultimately become disconnected from development, which is the opposite of the CSE thinking.

5.2.3 Continuous Compliance Hinders the Access to User Feedback

We found that compliance with legal regulations reduces access to user data (data-related challenges). Ensuring compliance with relevant regulations, such as GDPR, is a general issue related to data-driven organizations (Barbala et al. 2023). But regulatory compliance is also a core component of CSE. Klotins et al. (2022) point out that regulatory practices are often at odds with continuous principles. Continuous compliance is thus a cross-cutting concern that relates to the whole CSE process (see Fig. 2, white circle). In line with this, our study identified insufficient access to user data due to data privacy as one of the challenges related to user feedback in CSE.

Indeed, various sources of user data (demographic, usage data) offer enormous potential for product developers in continuous software engineering while at the same time seeming easily available as many companies and services tend to collect these data automatically. At the same time, access to these data is often reduced due to legal issues, ethical constraints (Yaman et al. 2020) and technical infrastructure issues (Lindgren and Münch 2016). When data cannot be accessed, certain types of user feedback cannot be collected and analyzed, thus reducing the feedback potential.

5.2.4 Continuous Monitoring Requires Continuous Validation of the Key Metrics

Our study indicated that choosing and maintaining meaningful metrics is a challenge for many. These metrics are necessary, among other things, for establishing infrastructure for mining event data. From the P5 case, we learned that finding the right metrics requires long-term experimentation when product developers continuously eliminate the metrics that do not make sense and add the ones that help understand the product dynamics.

This issue can be related to continuous monitoring, which is a practice of collecting and analyzing data on how the product is used and performs in a live environment, including metrics from the CI/CD pipeline (Klotins et al. 2022). This data is collected by applying implicit user data techniques (e.g., event logs, demographic data, etc.) with various metrics. Identifying metrics to evaluate the value and success of the product is often regarded a challenge in continuous experimentation (Lindgren and Münch 2016). In addition, our study also demonstrates that identification is such metrics must be performed on continuous basis to ensure that the product is being monitored properly.

To achieve such continuous monitoring, developers in some cases were using telemetry dashboards. As an informant from case P5 explained:

> *"the dashboard can't be static when the product, the business and the customer change all the time."*

Although telemetry dashboards have the potential to make software engineering truly continuous and data driven, we also found numerous challenges associated with such tools.

First, developing a telemetry dashboard requires significant investments related to establishing the infrastructure (accessing, uniforming, and representing the relevant user data) and acquiring specific competence (e.g., data scientists) over time. This investment can be decreased if the dashboard is based on an off-the-shelf tool (such as Amplitude, Google analytics). Also, the up-front investments into a dashboard may pay off over time because as the number of end-users increases, the feedback quality (generalizability) also rises without additional costs. However, adopting such solutions still involves financial costs, as indicated by the informant in case P5.

Second, the precondition for using telemetry dashboards is, of course, access to telemetry, specifically, to implicit user feedback. This makes the dashboards unavailable for early-stage products where the implicit feedback may not be accessible (e.g., due to lack of automated user data mining) or unrepresentative (because of insufficient number of users).

Third, achieving full potential from the use of a telemetry dashboard requires continuous validation and adjusting the metrics that are being monitored (as indicated by one of the challenges). However, in CSE, where so many other activities must be performed continuously (e.g. operations, development, feedback collection, planning, etc.), maintaining a telemetry dashboard up to date introduces additional strain. Furthermore, it is unclear whether the off-the-shelf dashboard tools offer sufficient flexibility and can be adjusted to the product needs in the same way as the self-developed dashboards.

An interesting question for future research is whether the number of metrics influences the quality of continuous monitoring. We saw that the products varied in terms of the number of metrics they had available to monitor their product. It is intuitive that products with a high number of available metrics have access to higher-quality feedback. However, as we see from the example of P9, numerous metrics may also introduce the issue of interpretability. There-

fore, we assume that not the number of metrics, but their selection is the key to continuous monitoring success.

In summary, continuous monitoring is a challenge in CSE that can be addressed by utilizing telemetry. Such dashboards may improve the flow between user feedback capture, product monitoring, and subsequent planning of technical activities. However, they also introduce issues, such as increased costs, unavailability of early stage products, and the need to be continuously maintained. Since the use of telemetry dashboards in CSE is not explored sufficiently, we suggest this as a subject for future research.

5.2.5 Insufficient Data Literacy May Reduce the Value of Continuous Feedback

Another finding suggests that product developers are sometimes dissatisfied with the rigorousness with which they collect or analyze user feedback (method-related challenges). The reasons for this were not always clear, making it interesting to further explore why product developers choose to make shortcuts while working with user feedback. In many cases, "shortcuts" were motivated by insufficient resources. However, this issue alone could not explain the lack of rigor in all cases.

The lack of analytical expertise has been reported to be one reason why the feedback is not sufficiently analyzed (Lindgren and Münch 2016). Indeed, if practitioners do not master the ways to analyze the feedback, they may either skip the analysis step or simply prioritize the data that are simplest for them and others to interpret.

If software development is to become truly data-driven, data literacy seems to be a logical precondition. In CSE, understanding what user data may and may not imply should influence important decisions related to product planning. We therefore suggest that data literacy is a cross-cutting concern in CSE, see Fig. 2.

5.3 Implications for Practice

Our study proposes a 1) conceptual model of how user feedback is handled in CSE and 2) an overview of the potential challenges. This knowledge can benefit real-life software development projects by informing on the practical steps of how user feedback can be integrated into CSE (such as defining the goal, picking appropriate techniques, using analysis in decision support), and by helping avoid or better understanding the reasons behind the reported pitfalls.

We will now discuss two specific takeaways that we believe are especially relevant to CSE practitioners.

5.3.1 Teams and Managers Should Strive for More Thorough Analysis of User Feedback to Facilitate Product Planning

One of the main takeaways of this study is that user feedback in CSE may not be adequately analyzed. CSE is made possible by increasing the availability of user data and automation, which increases the speed and frequency of releases. However, although a large part of user data (feedback acquisition) may be automated (for example, through data mining), feedback interpretation is not easy to automate. Of course, automated analysis of implicit feedback, such as A/B tests at Microsoft (Fabijan et al. 2017), is well known. However,

such implicit feedback is used mainly to validate requirements and bug fixes, not to discover new features (Johanssen et al. 2019). Therefore, the interpretation of explicit feedback is also important. However, if it cannot be automated, it inevitably compromises the process of CSE, either by slowing down the release speed or frequency; or by reducing the feedback quality. Additionally, we found that user feedback may not be analyzed due to low data literacy (Section 5.2.5). This leaves the problem of feedback interpretation and analysis unsolved. The lack of sufficiently analyzed user feedback, in turn, disconnects user insight from product planning, thus limiting the benefits of adopting CSE.

To address this problem, data analysis and data literacy should become a much stronger focus of product development teams. This can be achieved by allocating additional time for analysis of user feedback as opposed to developing small fixes based on low hanging fruit. **Including discussions of user feedback in the team planning event (e.g., daily meetings) can facilitate the necessary focus on understanding the feedback and making decisions).** Such discussions can be facilitated by team members or coaches who are sometimes responsible for team meetings (Stray et al. 2021).

For product managers, data analysis should also become a greater focus. **They should strive to increase their own data literacy, helping teams make better decisions based on user feedback.** Previous results show that when performed in an agile way, the role of product manager can contribute to better integration between business goals, user feedback and developer teams, thus contributing to better integration of BizDev and the whole CSE cycle. Tkalich et al. (2022).

Both product managers and teams should ensure access to the necessary analytic competence. This competence can be enhanced by involving data analysis experts. We saw that in the cases where data analysts were active (P5, P7), the teams worked with user feedback more systematically and continuously. However, this should be done with caution, as collaboration between developers and data scientists is not straightforward (Hukkelberg and Berntzen 2019).

5.3.2 Managers and Product Developers Alike Should Invest in Development of Telemetry Dashboards and Continuously Adapt Them to the Needs of the Products

Another key finding of this study is that telemetry dashboards appear to play a crucial role in supporting CSE, but are often not used systematically or not prioritized at all. Telemetry dashboards are of significant importance in CSE because they contribute to several concepts: continuous monitoring, continuous product planning, and continuous improvement (see Subsection 5.2.4). Furthermore, such tools can be continuously adjusted, which ensures their relevance at all times (Section 5.1).

Therefore, we urge product developers to take into account the lessons learned from this study and incorporate telemetry dashboards into their practice if this is not yet in place. **The dashboards should be utilized in a way that allows to continuously adjust the metrics, thus ensuring their relevance for the product at all times.** One way to do this, which we saw in our study, was to develop a dashboard from scratch, which increases the chances that the dashboard is tailored to the needs of the team or the product.

Furthermore, **we encourage managers to provide resources for product developers to create and use telemetry dashboards**. One of the challenges that we identified was that

developers often considered establishing a telemetry dashboard too costly or did not have the competence to do so. In many cases, access to the relevant resources and competence can be facilitated by higher management.

5.4 Threats to Validity

We evaluated our study in terms of the four aspects of validity suggested by Runeso and Höst (2009).

Construct Validity relates to how the main concepts of our study (such as CSE and its constituents) represent what we intended to study (Runeso and Höst 2009). Specifically, whether our interview participants interpreted our questions correctly when we are asking about the CSE application in their products, etc.

To mitigate this threat, we explained what we mean by CSE in the introduction to the interviews. We also chose the products based on whether they were relying on CSE (inclusion criteria). Further, we validated with the research team that the products actually match our understanding of CSE. Finally, we asked our informants for feedback on the preliminary description of the results to validate that the concepts we are using match their understanding of their products.

Internal Validity refers to causal relationships inferred from the study (Runeso and Höst 2009). This threat is relevant for this paper because we summarize several conclusions based on our results and previous work, explaining how various aspects of CSE may influence the utilization of user feedback. In other words, our results suggest that CSE and user feedback practices and challenges may be related (for example, enhanced analysis of user feedback may improve the results of CSE). We base these suggestions on our data analysis, method and the state-of-the-art. However, the suggestions need to be validated by future work.

External Validity reflects how our findings can be relevant outside of our study, or how generalizable they are Runeso and Höst (2009). The specific threats here are mostly related to sampling. Since we were recruiting the informants based on our connections and their references (see the section on data collection) our results might not be readily generalizable. Further, each case is represented by accounts of 1 to 3 informants, which creates an imbalance of accounts per case. Additionally, our cases originate from a particular cultural context (all of the companies are based or operate in Norway), and can thus differ from cases in other cultures. Finally, the number of respondents (19) and cases (13) is relatively small which reduces the generalizability of the findings. To mitigate this threat, we rely on the state-of-the-art to support the conclusions that we draw from data analysis.

We tried to increase external validity by presenting the contextual details of the cases studied, comparing our findings with the current literature, and highlighting any unique findings. We see that many of our findings are in line with earlier research.

Reliability The potential threat concerns replicability of the results by others (Runeso and Höst 2009). This study is based on ongoing work with a number of industrial partners. Their context, challenges, product maturity stages, etc. cannot be easily replicated. Thus, the study context remains unique. To support the replication of our study in another context we share our interview guide, details of data analysis, and the coding scheme for the cross-case

analysis. Due to confidentiality agreements with our partners, we cannot publish raw data, e.g. full interview transcripts, but they are available at a reasonable request.

When it comes to data analysis itself, the technique we applied (thematic analysis) originates in the social sciences (Braun and Clarke 2006). This may threaten the reliability because the data material was collected in the software engineering context and is thus sociotechnical in nature. However, social science research has paved the way for SE research, with thematic analysis being applied in numerous studies, including those with similar data (Cico et al. 2020; Saad et al. 2021). Therefore, we believe that the application of thematic analysis in our study is justified.

5.5 Future Work

First, additional insight is needed about the effect of product context on practices related to user feedback. In this study, we focus on general trends related to user feedback in CSE and also try to explore whether context-specific characteristics can affect general observations. For example, our findings suggest that in products that aim for niche (as opposed to general) user groups, product developers may more often seek explicit feedback, such as ethnographic-style observations because these techniques allow one to better solve complex user problems. However, a larger case sample is needed to explore this and similar observations relating to such factors as company size, company culture, market type, etc.

Second, it is interesting to explore best practices for user sampling when it comes to continuous feedback. We found that convenience sampling can be used as user feedback techniques; however, we did not focus on how user sampling is performed in general and which strategies provide the best results.

Third, we need further knowledge on how telemetry dashboards and similar tools are used in CSE. In this study, we described the benefits and challenges of such dashboards and concluded that they can improve feedback-related practices at several stages of the CSE process. However, such tools are not well-explored in CSE.

6 Conclusions

Although significant knowledge has been built on user feedback in software engineering in general (including lean and agile software development), understanding of the related processes in CSE remains insufficient. Therefore, the purpose of this study was to explore how user needs and preferences are inferred in CSE and what the related challenges are. Based on the results reported from 13 industrial cases and our previous work, we have suggested a conceptual model that describes the process of capturing, analyzing, and applying the insight on end-users in CSE.

Our findings indicate that practitioners utilize feedback techniques that provide explicit and implicit user insight. In our cases, though limited in number, we have seen that, most often, practitioners seek explicit (qualitative) insight to solve user problem and implicit (qualitative) insight to assess product-market fit. However, further analysis and use of feedback for product planning may be insufficient or lacking due to release pressure and ever-increasing volumes of feedback from various sources. Further, working with user feedback in CSE is challenging because of long-term commitment to the end-users, inflexible resource allocation, the need to comply with data-related regulations, to choose the right metrics; and, finally, due to insufficient data literacy.

Based on these findings, we warn practitioners that insufficient analysis of user feedback can undermine the effort to collect feedback. One way to improve the analysis and use of various sources of feedback is through telemetry dashboards, which have the potential to enhance the flow between capturing user feedback and planning technical activities based on the feedback.

Appendix I

Here we provide contextual details of the studied companies and products (cases).

Studied Products (cases) with Respective Companies

Real names of the products and companies are shown in parenthesis (whenever available).

Cases at *Company A* (Iterate)

Product 1/P1 (Vake) - Vake is a cutting-edge AI-enabled maritime surveillance tool. One of the internal startups developed within the innovation ecosystem at Iterate. Designed to cater to both corporate and public customers, the product aims to revolutionize maritime surveillance with advanced technology. At the time of the data collection, the product team is focusing on developing the core functionality, which requires close collaboration with a small selection of pilot users. Findings on this product are earlier reported in Tkalich et al. (2021a).

Product 2/P2 (Hjernelæring) - Hjernelæring is a platform that enables teachers and parents to openly engage in conversations about kids' emotions in their daily lives. The aim is to support the growth of resilience in children. With over 2000 teachers piloting the school curriculum, it is creating value for kids as well as teachers and parents. Now, the team is trying to understand how to monetize the concept.

Product 3/P3 (Unicad) - Unicad is an online calculation tool for construction engineers. Currently, the product has four pilot users who are in close contact with the product team. Earlier findings on this product are published in Tkalich et al. (2021a, b).

Product 4/P4 (*Byks*) - Byks is a training platform where users may follow training programs created by professional athletes. The product team's main focus at the time of data collection is the retention of about 170 existing users of the training (B2C customers) and customizing the platform according to the preferences of a few key athletes (B2B customers).

Product 5/P5 (Flow Technologies AS) - a platform providing rehabilitation for patients of health institutions in several countries. The product holds contracts with 11 health providers and works closely with only a few of them to refine and develop the functionality offered to a total of about 8000 patients. Other results on this product are published in Tkalich et al. (2021a).

Product 6/P6 (Noba) - Noba is an innovative tool that assists people in evaluating the level of individual food safety based on its FODMAP[2] index. With more than 67 thousand downloads at Google Play and App Store, the team's current goal is to secure a revenue stream to be able to fully sustain the product and create profit.

Cases at *Company B (SpareBank 1 Utvikling)*

Product 7/P7 - an online banking solution where the team's main focus is incremental improvement and innovation of the existing overview page for private customers. The solution has numerous users across Norway.

Cases at *Company C*

Product 8/P8 - tailor-made software solution specifically designed for governmental case-workers, facilitating the efficient processing of applications related to health-related benefits. With a focus on user-centricity, P8 regularly releases new functions and actively collects feedback from caseworkers to optimize the software's alignment with their workflow.

Product 9/P9 - an established platform for job seeking and recruitment developed by a public company to support job seekers and employers. Despite having a well- functioning website, the team is continuously searching for new ways of connecting job seekers with employers, aiming to provide an even more comprehensive and effective job-seeking platform.

Cases at Other Companies

Product 10/P10 (Emissions Insights), Company D (DNV) - A dashboard for operators of maritime vessels showing an overview of the fleet's CO2 emissions. The product is part of a larger product family for the same customer group and had been recently developed to differentiate DNV from the competitors. Findings on this product have been reported in Tkalich et al. (2021b)

Product 11/P11, Company E - An online journey planner with an impressive track record of over 100 thousand downloads from the Google Play Store. The dedicated team behind P11 takes responsibility for the continuous incremental improvement of this tool. As the core product of the respective company, the online journey planner benefits from the collective efforts of various teams, each specializing in different functional aspects. Other findings on this product are also reported in Sporsem and Moe (2022)

Product 12/P12 (Sandstorm), Company F (Zedge) - An online meme generator that recently reached a million installs at Google Play Store. The product manager is currently the only member of the product team who is focusing on increasing the product's usability and profitability (part-time) while working as a full-time developer and product manager at Zedge.

[2] FODMAP is an acronym used to describe a group of carbohydrates that can cause irritable bowel.

Product 13/P13, Company G - An online marketplace for selling and ordering interior design solutions. The platform is presently utilized for a selection of 12 interior designers who advertise and sell their services to about 300 B2C customers per year.

Studied Companies

The real names of the companies are shown in parentheses (whenever available).

Company A (*Iterate*) - Iterate is a medium-sized software and investment company with 80 employees where the products are integrated into an innovative ecosystem, sharing infrastructure, office space, and events. Product managers work part-time on their respective products until they reach maturity, while also serving as consultants for the company during the remaining time.

Company B (SpareBank 1 Utvikling) - an alliance of banks with an increased focus on digitalization and software development. The company has 6300 employees. Striving to stay at the forefront of technological advancements, the company places considerable focus on enhancing digital capabilities within the banking industry. Additional details about the company are reported in Tkalich et al. (2022); Moe et al. (2023).

Company C - A public welfare organization with 22000 employees providing a wide range of essential services to citizens. The company plays a crucial role in promoting social welfare and supporting individuals in various aspects of their lives, such as employment, social security, and healthcare.

Company D (DNV) - A maritime service company that has 3700 employees and a global presence. The primary focus is digital transformation and software innovation.

Company E - A public-owned company, employing 250 individuals, that specializes in providing digital services within the transportation sector. Their focus is on leveraging technology to enhance transportation experiences and efficiency. More information on the company can be found in Berntzen et al. (2023).

Company F (Zedge) - A globally-distributed company with 80 employees, dedicated to developing content distribution software tailored for mobile devices. delivery on mobile platforms.

Company G - A small provider of interior design services operating via an online platform. The company has 10 employees.

Acknowledgements The Research Council of Norway partially supported the work through the projects 10xTeams (grant 309344), Transformit (grant 321477), and Digital Class (grant 309631).
The contribution by Eriks Klotins is supported by Swedish Knowledge Foundation (KK-stiftelsen) projects SERT Profile (Ref. 2018/010) and "Cost Benefit Analysis of Continuous Software Engineering" (Ref. 202100/4011)

Funding Open access funding provided by SINTEF.

Data Availability The datasets generated during and/or analyzed during the current study (transcripts of semi-structured interviews) are not publicly available due to anonymity concerns about companies and individual

research participants. Supplementary material showing the initial steps of the qualitative analysis (themes and underlying codes) is available online at: https://zenodo.org/records/12544288.

Declarations

Compliance with Ethical Standards All data collection was approved by Norwegian center for research data (NSD). All interview participants provided their informed consent.

Competing Interests The authors have no competing interests to declare that are relevant to the content of this article.

References

Alahyari H, Gorschek T, Svensson RB (2019) An exploratory study of waste in software development organizations using agile or lean approaches: A multiple case study at 14 organizations. Inf Softw Technol 105:78–94

Aleksander Fabijan, Helena Holmström Olsson, and Jan Bosch. Customer feedback and data collection techniques in software r&d: a literature review. In *Software Business: 6th International Conference, ICSOB 2015, Braga, Portugal, June 10-12, 2015, Proceedings 6*, pages 139–153. Springer, 2015

Aleksander Fabijan, Pavel Dmitriev, Helena Holmström Olsson, and Jan Bosch. The evolution of continuous experimentation in software product development: from data to a data-driven organization at scale. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, pages 770–780. IEEE, 2017

Anastasiia Tkalich, Darja Šmite, Nina Haugland Andersen, and Nils Brede Moe. What happens to psychological safety when going remote? *IEEE Software*, 2022

Anastasiia Tkalich, Nils Brede Moe, and Rasmus Ulfsnes. Making internal software startups work: how to innovate like a venture builder? In *International Conference on Software Business*, pages 152–167. Springer, 2021a

Anastasiia Tkalich, Nils Brede Moe, and Tor Sporsem. Employee-driven innovation to fuel internal software startups: preliminary findings. In *International Conference on Agile Software Development*, pages 145–154. Springer, 2021b

Anastasiia Tkalich, Rasmus Ulfsnes, and Nils Brede Moe. Toward an agile product management: What do product managers do in agile companies? In *International Conference on Agile Software Development*, pages 168–184. Springer, 2022

Astri Barbala, Tor Sporsem, and Viktoria Stray. Data-driven development in public sector: How agile product teams maneuver data privacy regulations. In *International Conference on Agile Software Development*, pages 165–180. Springer, 2023

Baltes S, Ralph P (2022) Sampling in software engineering research: A critical review and guidelines. Empir Softw Eng 27(4):94

Berntzen M, Stray V, Moe NB, Hoda R (2023) Responding to change over time: A longitudinal case study on changes in coordination mechanisms in large-scale agile. Empir Softw Eng 28(5):114

Björn Regnell and Sjaak Brinkkemper. Market-driven requirements engineering for software products. *Engineering and managing software requirements*, pages 287–308, 2005

Braun V, Clarke V (2006) Using thematic analysis in psychology. Qual Res Psychol 3(2):77–101

Brhel M, Meth H, Maedche A, Werder K (2015) Exploring principles of user-centered agile software development: A literature review. Inf Softw Technol 61:163–181

Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in software engineering*. Springer Science & Business Media, 2012

Dejana Bajic and Kelly Lyons. Leveraging social media to gather user feedback for software development. In *Proceedings of the 2nd international workshop on Web 2.0 for software engineering*, pages 1–6, 2011

Eisenhardt KM (1989) Building theories from case study research. Acad Manag Rev 14(4):532–550

Emitza Guzman, Mohamed Ibrahim, and Martin Glinz. Prioritizing user feedback from twitter: A survey report. In *2017 IEEE/ACM 4th International Workshop on CrowdSourcing in Software Engineering (CSI-SE)*, pages 21–24. IEEE, 2017

Eriks Klotins, Michael Unterkalmsteiner, Panagiota Chatzipetrou, Tony Gorschek, Rafael Prikladnicki, Nirnaya Tripathi, and Leandro Bento Pompermaier. A progression model of software engineering goals, challenges, and practices in start-ups. *IEEE Transactions on Software Engineering*, 47(3):498–521, 2019

Fitzgerald B, Stol K-J (2017) Continuous software engineering: A roadmap and agenda. J Syst Softw 123:176–189

Florian Auer and Michael Felderer. Current state of research on continuous experimentation: a systematic mapping study. In *2018 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 335–344. IEEE, 2018

Foundjem Armstrong (2023) Ellis E Eghan, and Bram Adams. Feedback diversity vs. synchronization. IEEE Transactions on Software Engineering, A grounded theory of cross-community secos

Fowler M, Highsmith J et al (2001) The agile manifesto. Softw Dev 9(8):28–35

Frauke Paetsch, Armin Eberlein, and Frank Maurer. Requirements engineering and agile software development. In *WET ICE 2003. Proceedings. Twelfth IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises, 2003.*, pages 308–313. IEEE, 2003

Gallivan MJ, Keil M (2003) The user-developer communication process: a critical case study. Inf Syst J 13(1):37–68

Ghasemi M, Amyot D (2020) From event logs to goals: a systematic literature review of goal-oriented process mining. Requir Eng 25(1):67–93

Harrie Jansen et al. The logic of qualitative survey research and its position in the field of social research methods. In *Forum Qualitative Sozialforschung/Forum: Qualitative Social Research*, volume 11, 2010

Inayat I, Salim SS, Marczak S, Daneva M, Shamshirband S (2015) A systematic literature review on agile requirements engineering practices and challenges. Comput Human Behav 51:915–929

Ivar Hukkelberg and Marthe Berntzen. Exploring the challenges of integrating data science roles in agile autonomous teams. In *Agile Processes in Software Engineering and Extreme Programming–Workshops: XP 2019 Workshops, Montréal, QC, Canada, May 21–25, 2019, Proceedings 20*, pages 37–45. Springer, 2019

Jalali S, Wohlin C (2012) Global software engineering and agile practices: a systematic review. J Softw Evol Process 24(6):643–659

Jan Bosch, Helena Holmström Olsson, and Ivica Crnkovic. It takes three to tango: Requirement, outcome/data, and ai driven development. In *SiBW*, pages 177–192, 2018

Jan Ole Johanssen, Anja Kleebaum, Bernd Bruegge, and Barbara Paech. How do practitioners capture and utilize user feedback during continuous software engineering? In *2019 IEEE 27th International Requirements Engineering Conference (RE)*, pages 153–164. IEEE, 2019

Jez Humble and Gene Kim. *Accelerate: The science of lean software and devops: Building and scaling high performing technology organizations*. IT Revolution, 2018

Johan Linaker, Sardar Muhammad Sulaman, Martin Höst, and Rafael Maiani de Mello. *Guidelines for conducting surveys in software engineering*. Lund University, 2015

Khomh F, Adams B, Dhaliwal T, Zou Y (2015) Understanding the impact of rapid releases on software quality: The case of firefox. Empir Softw Eng 20:336–373

Klotins E, Unterkalmsteiner M, Chatzipetrou P, Gorschek T, Prikladnicki R, Tripathi N (2019) Pompermaier LB A progression model of software engineering goals, challenges, and practices in start-ups. IEEE Trans Softw Eng 47(3):498–521

Klotins E, Gorschek T, Sundelin K, Falk E (2022) Towards cost-benefit evaluation for continuous software engineering activities. Empir Softw Eng 27(6):157

Lindgren E, Münch J (2016) Raising the odds of success: the current state of experimentation in product development. Inf Softw Technol 77:80–91

Maalej W, Nayebi M, Johann T, Ruhe G (2015) Toward data-driven requirements engineering. IEEE Softw 33(1):48–54

Maalej Walid, Nayebi Maleknaz, Johann Timo, Ruhe Guenther (2015) Toward data-driven requirements engineering. IEEE software 33(1):48–54

Mark Crowne. Why software product startups fail and what to do about it. evolution of software product development in startup companies. In *IEEE International Engineering Management Conference*, volume 1, pages 338–343. IEEE, 2002

Mary Poppendieck and Tom Poppendieck. *Lean Software Development: An Agile Toolkit: An Agile Toolkit*. Addison-Wesley, 2003

Melegati J, Edison H, Wang X (2020) Xpro: a model to explain the limited adoption and implementation of experimentation in software startups. IEEE Trans Softw Eng 48(6):1929–1946

Melegati J, Guerra E, Wang X (2021) Understanding hypotheses engineering in software startups through a gray literature review. Inf Softw Technol 133:106465

Nicole Forsgren, Dustin Smith, Jez Humble, and Jessie Frazelle. 2019 accelerate state of devops report, 2019

Nils Brede Moe, Viktoria Stray, Darja Smite, and Marius Mikalsen. Attractive workplaces: What are engineers looking for? *IEEE Software*, 2023

Orges Cico, Anh Nguyen Duc, and Letizia Jaccheri. An empirical investigation on software practices in growth phase startups. In *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering*, pages 282–287, 2020

Pilar Rodríguez, Jouni Markkula, Markku Oivo, and Juan Garbajosa. Analyzing the drivers of the combination of lean and agile in software development companies. In *International Conference on Product Focused Software Process Improvement*, pages 145–159. Springer, 2012

Product Plan. Aarrr pirate metrics framework, 2024

Robert C Martin. *Agile software development: principles, patterns, and practices*. Prentice Hall, 2002

Ros R, Bjarnason E, Runeson P (2024) A theory of factors affecting continuous experimentation (face). Empir Softw Eng 29(1):21

Runeson P, Höst M (2009) Guidelines for conducting and reporting case study research in software engineering. Empir Softw Eng 14:131–164

Saad J, Martinelli S, Machado LS, de Souza CR, Alvaro A, Zaina L (2021) Ux work in software startups: A thematic analysis of the literature. Inf Softw Technol 140:106688

Sidky A, Arthur J, Bohner S (2007) A disciplined approach to adopting agile practices: the agile adoption framework. Innov Syst Softw Eng 3(3):203–216

Stray V, Tkalich A, Moe NB. The agile coach role: Coaching for agile performance impact. In *Proceedings of the Annual Hawaii International Conference on System Sciences (HICSS)*. University of Hawai'i at Manoa, 2021

Tor Sporsem and Nils Brede Moe. Coordination strategies when working from anywhere: a case study of two agile teams. In *International Conference on Agile Software Development*, pages 52–61. Springer, 2022

Tor Sporsem, Morten Hatling, Anastasiia Tkalich, and Klaas-Jan Stol. Knowns and unknowns: An experience report on discovering tacit knowledge of maritime surveyors. In *International Working Conference on Requirements Engineering: Foundation for Software Quality*, pages 309–323. Springer, 2023

Xavier Franch, Aron Henriksson, Jolita Ralyté, and Jelena Zdravkovic. Data-driven agile requirements elicitation through the lenses of situational method engineering. In *2021 IEEE 29th International Requirements Engineering Conference (RE)*, pages 402–407. IEEE, 2021

Yaman S, Fagerholm F, Munezero M, Männistö T, Mikkonen T (2020) Patterns of user involvement in experiment-driven software development. Inf Softw Technol 120:106244

Authors and Affiliations

Anastasiia Tkalich[1] · Eriks Klotins[2] · Tor Sporsem[1] · Viktoria Stray[1,3] · Nils Brede Moe[1,2] · Astri Barbala[1]

✉ Anastasiia Tkalich
anastasiia.tkalich@sintef.no

Eriks Klotins
eriks.klotins@bth.se

Tor Sporsem
tor.sporsem@sintef.no

Viktoria Stray
stray@ifi.uio.no

Nils Brede Moe
nils.b.moe@sintef.no

Astri Barbala
astri.barbala@sintef.no

1 SINTEF, Trondheim, Norway

2 Software Engineering Research Lab (SERL), Blekinge Institute of Technology, Karlskrona, Sweden

3 University of Oslo (UiO), Oslo, Norway